



OPEN Task offloading for multi-server edge computing in industrial Internet with joint load balance and fuzzy security

Xiaomin Jin^{1,2,3}✉, Shuai Zhang^{1,2,3,4}, Yurong Ding^{1,2,3,4} & Zhongmin Wang^{1,2,3,4}

The industrial Internet revolutionizes traditional manufacturing through the incorporation of technologies such as real-time production optimization, big data analysis, etc. Computing resource-constrained industrial terminals struggle to effectively execute latency-sensitive and computation-intensive tasks triggered by these technologies. Edge computing (EC) emerges as a promising paradigm for offloading tasks from terminals to the adjacent edge servers, offering the potentiality to augment the computational capacities for industrial terminals. However, the development of accurate offloading strategies poses a prominent challenge for EC in the industrial Internet. Incorrect offloading strategies will misguide the task offloading procedure, resulting in adverse consequences. In this paper, we study the latency-aware multi-server partial EC task offloading problem in the industrial Internet with the consideration of joining load balancing and security protection to provide accurate strategies. Firstly, we establish a task offloading model that supports partial offloading, facilitating latency reduction, task offloading across multiple edge servers with load balance, and accommodation of fuzzy task risks. We quantify the established model as a constrained optimization formulation and prove its NP-hardness. Secondly, to solve the composite offloading strategy comprising both the offloading location and offloading ratio derived from our model, we propose a bi-layer offloading algorithm with joint load balance and fuzzy security, which is based on the adaptive genetic algorithm and simulated annealing particle swarm optimization. Based on extensive experimental results, we find that the established model is effective in reducing the objective value, with a respective decrease of 27% and 46% compared to full execution in edge servers and local execution in industrial terminals. Furthermore, the proposed offloading algorithm exhibits superior performance in terms of solution accuracy compared to existing algorithms.

Keywords Industrial Internet, Edge computing, Task offloading, Bi-layer offloading algorithm

The industrial Internet serves as a cornerstone for the next industrial revolution by elevating the intelligence level of traditional manufacturing industries^{1,2}. The industrial Internet necessitates strong computing power as a support to meet the challenges of real-time product optimization, large-scale data processing, intelligent decision-making, etc. The primary focus of industrial terminals is manufacturing, and their computing power is relatively weak due to constraints such as space, heat dissipation, and cost, which results in insufficient computational capabilities to support the industrial Internet adequately. Several computing paradigms, such as cloud computing, fog computing, and edge computing (EC), have been developed to serve as computing platforms that offer computational resources for impoverished terminals. Task offloading enables the migration of tasks to external platforms, resulting in the enhancement of terminal capabilities³. Unfortunately, as a mature computing paradigm, cloud computing fails to address the computational challenges faced by the industrial Internet due to its inherent characteristics of long-backhaul latency, lower security in data transmission through public networks, and significant bandwidth consumption⁴. Fog computing utilizes computing power on network nodes for processing during data transmission, necessitating transmission over public networks and where the computational capabilities of network nodes are very limited⁵. Edge servers are typically positioned within a

¹Xi'an University of Posts and Telecommunications, School of Computer Science and Technology, Xi'an 710121, China. ²Shaanxi Key Laboratory of Network Data Analysis and Intelligent Processing, Xi'an 710121, China. ³Xi'an Key Laboratory of Big Data and Intelligent Computing, Xi'an 710121, China. ⁴Shuai Zhang, Yurong Ding and Zhongmin Wang contributed equally to this work. ✉email: xmjin@xupt.edu.cn

one-hop distance from industrial terminals, offering the potentiality for low latency, bandwidth efficiency, and enhanced security⁶. EC provides a compliant computational platform for the industrial Internet, enabling the offloading of industrial computing tasks to edge servers⁷.

The decision regarding which tasks should be offloaded to edge servers has a significant impact on the EC performance and is the primary challenge that EC must address⁸. The partial offloading allows tasks to be proportionally offloaded to edge servers for execution, providing a finer granularity compared to binary offloading (the task is either fully offloaded or not offloaded at all). Improper task offloading not only hinders the full potentiality of EC but also suppresses its performance due to the additional data transmission overhead, ultimately rendering EC incapable of supporting the industrial Internet⁹.

Additionally, latency and security constitute two pivotal requirements for EC task offloading in the industrial Internet. Many manufacturing processes necessitate low latency for real-time processing of their data^{10,11}. For example, in the manufacturing process of large-scale precision mechanical components, such as ship propellers, lathes dynamically adjust their parameters based on the shape and size of the components in real-time. The prolonged latency would impede the timely adjustment of cutting parameters on lathes, leading to the inability to achieve the desired specifications of the components and resulting in defective products. EC in the industrial Internet faces elevated security challenges as a result of the inclusion of industrial computing tasks, which frequently involve trade-sensitive information, such as mechanical size parameters and material recipes. Concerns regarding the possible disclosure of trade secrets may lead factory owners to exhibit hesitancy in adopting EC. Furthermore, EC entails the distributed deployment of a substantial number of edge servers, which inherently possess limited resources compared to cloud data centers. The isolated utilization of a single edge server falls short of fully leveraging the potentiality of EC. To fully exploit the benefits of EC, it is imperative to employ multiple edge servers in a collaborative manner¹². Therefore, EC task offloading in the industrial Internet is characterized by the coupling of latency, security, and multi-server considerations.

When considering multiple edge servers, the importance of load balancing cannot be overstated. Without proper load balancing, there is a risk of tasks from terminals being disproportionately offloaded to specific edge servers, resulting in an uneven distribution of load and the potentiality for network congestion among edge servers. Consequently, this imbalance can adversely impact the overall load distribution and resource utilization efficiency of the entire system, leading to a decrease in system throughput^{13–15}. The detrimental consequences of performance degradation or service interruption due to server overload can be effectively mitigated through the equitable distribution of tasks among multiple edge servers.

Furthermore, in real-life industrial Internet scenarios, assessing task security risks often involves multiple uncertainties, such as data gaps, subjective judgments, and fuzzy boundaries^{16,17}. Traditional binary evaluation methods provide only two discrete outcomes: risky or risk-free. This simplistic categorization may overlook the complexity and subtle variations of task risks, leading to potential information loss in the evaluation process. Task risks are typically continuous and progressive, and binary methods fail to capture these nuances accurately. In practice, industrial task risks often exist at varying levels and degrees, making binary classification insufficient. Fuzzy numbers provide a more flexible and adaptive approach to handling the uncertainty and variability of task risks¹⁸. The fuzzy evaluation method allows for customization based on specific circumstances and requirements by adjusting the parameters and boundaries of fuzzy numbers. Compared to traditional binary methods, fuzzy numbers offer a more comprehensive spectrum of risk levels, enabling a more accurate representation of task risk gradations. Through fuzzy sets and membership functions, fuzzy logic can better express the various levels and subtle variations in task risks, providing a more precise and adaptable evaluation method.

The motivation of this paper is to tackle the latency-aware multi-server partial EC task offloading problem in the industrial Internet, emphasizing the integration of joint load balancing and security protection to optimize resource allocation and ensure the confidentiality of sensitive industrial data. To achieve this goal, we present an innovative task offloading approach that includes a novel task offloading model and a novel bi-layer offloading algorithm. To formulate the task offloading problem, we establish a task offloading model that enables partial offloading, leading to reduced latency, task offloading across various edge servers with load balancing, and consideration of fuzzy task risks. The established model is formulated as a three-objective constrained mathematical optimization model that targets the reduction of latency, load balancing, and mitigation of security risks. After formalizing the problem, we provide a proof demonstrating that the formulated model is NP-hard. Unfortunately, due to the inherent complexity of NP-hard problems, exact solutions cannot be obtained in polynomial time. Nevertheless, an alternative approach using evolutionary algorithms can be employed to efficiently approximate solutions. Evolutionary algorithms, drawing inspiration from natural processes such as human activities, swarm intelligence, and physical phenomena, exhibit the capacity to efficiently identify global optimum solutions. This efficacy is characterized by reduced computational time, a harmonious blend of exploration and exploitation, and the delivery of precise results^{19–21}. Therefore, we propose a bi-layer offloading algorithm with joint load balance and fuzzy security, which is based on the adaptive genetic algorithm and simulated annealing particle swarm optimization, to solve the established model. Our offloading algorithm decomposes the established model into two layers for solution. The upper-layer algorithm determines which edge server the task should be offloaded to, while the lower-layer algorithm determines the ratio of task offloading. The main contributions of this paper are summarized as following:

- (1) We establish a latency-aware partial task offloading model for multi-server EC in the industrial Internet with joint load balancing and security protection. The established model supports fine-grained multi-server task offloading, allowing tasks to be offloaded proportionally. Moreover, this model employs fuzzy numbers to represent task risks and balances the load among multiple edge servers. We formulate this model as a constrained mixed integer optimization problem and prove its NP-hardness.

- (2) We propose a bi-layer offloading algorithm with joint load balance and fuzzy security, which is based on the adaptive genetic algorithm and simulated annealing particle swarm optimization, to solve the offloading strategy from the established model. Our offloading algorithm decomposes the model into two layer for solution, and constructs the offloading strategy as a composite of offloading location and ratio. We employ the adaptive genetic algorithm in the upper-layer algorithm to determine the offloading location, and utilize simulated annealing particle swarm optimization in the lower-layer algorithm to optimize the offloading ratio based on the obtained offloading location.
- (3) We conduct extensive experiments to validate the established offloading model and evaluate the effectiveness of the proposed offloading algorithm. These experiments comprehensively investigate the influence of diverse system parameters on the optimization objective. Additionally, we perform a comparative analysis of our offloading algorithm against five existing algorithms to assess its efficiency and performance. The remainder of this paper is organized as follows. The second section summarizes the related work. The third section describes the system model, the problem formulation, and the proof of NP-hardness. The fourth section illustrates the proposed offloading algorithm. In the fifth section, the established model and proposed offloading algorithm are evaluated. Finally, the sixth section concludes this paper.

Related work

As a new paradigm that can support the industrial Internet, EC integrates various domains including communication, Internet of things (IoT), and automation, and has garnered significant attention as a prominent research area. Task offloading, as a primary challenge in the realm of EC, has garnered considerable attention and witnessed substantial research endeavors. We categorize existing relevant work into the following three classes for summarization and analysis.

Basic task offloading

You and Tang²² studied the task offloading problem in industrial IoT and proposed a particle swarm optimization-based task offloading strategy. They employed their strategy to offload tasks from resource-constrained terminals to edge servers, with the objective of reducing latency, energy consumption, and task execution cost. Zhou et al.²³ studied the task offloading problem in the healthcare IoT and formulated the offloading problem as an adversarial multi-armed bandit model. They proposed a ultra reliable and low latency communication-aware scheme to minimize the long-term energy consumption under latency requirements. Jiao et al.²⁴ studied the the online offloading problem in industrial IoT to optimize the tradeoff of the task accomplishing time and energy consumption. They proposed a time-energy trade-off online offloading algorithm called TETO to solve the online offloading problem. Deng et al.²⁵ proposed two partial offloading strategies based on the reinforcement learning to optimize the latency in industrial IoT. Liu et al.²⁶ studied the task offloading problem in the ultra-dense network and formulated this problem as a Markov decision process model that aims to minimize the latency and energy consumption in the long term. They proposed an attention-based double deep Q network to solve their offloading model. Dai et al.²⁷ investigated the collaborative task offloading problem to achieve latency reduction while avoiding network congestion. They proposed a collaborative task offloading framework and a learning-based algorithm for collaborative task offloading to minimize the system cost (i.e., task latency and offloading cost). Guo et al.²⁸ studied the task offloading problem in the industrial Internet and formulated it as a mixed integer non-linear programming model. They proposed a heuristic algorithm based on greedy policy to minimize the system overhead caused by task offloading. Saleem et al.²⁹ provided an integrated partial offloading and interference management framework using the orthogonal frequency division multiple access scheme. They formulated the task offloading problem as a mixed integer nonlinear programming model and proposed a scheme named JPORA to minimize the task execution latency. Sun et al.³⁰ proposed a Lyapunov-based task offloading algorithm for 5G-enabled Internet of vehicles scenarios to maximize the average profit of mobile EC providers through optimal task offloading decisions. Sun et al.³¹ addressed the task scheduling problem in space-air-ground integrated networks and proposed an algorithm named PFAPPO that combines proportional fairness-aware auction with deep reinforcement learning to optimize resource allocation and task offloading, thereby maximizing the system profit and ensuring load balancing. Sun et al.³² proposed a game-theoretic framework for joint mode selection and power adaptation in 5G vehicular networks to maximize the network throughput and improve energy efficiency while meeting the delay requirements of multi-priority vehicular services.

Task offloading with joint load balance

Mondal et al.³³ proposed an economic and non-cooperative game-theoretic model for load balancing among competitive cloudlets. They formulated the problem as a generalized Nash equilibrium model and designed a variational inequality based algorithm to compute the pure-strategy Nash equilibrium. Chen et al.³⁴ studied the task offloading problem with joint load balancing in the ultra-dense network and proposed a load estimation algorithm based on user load prediction. Xu et al.³⁵ studied the edge server selection and task allocation problem, and formulated this problem as an online linear programming model with the objective of minimizing task execution and transmission make-spans. They proposed an online deep reinforcement learning-based simultaneous multi-server offloading algorithm to solve their model. Lu et al.³⁶ proposed a workload balancing scheme for multi-road side units, which dynamically adapts to varying task popularity in dynamic environments. This scheme aims to prevent resource wastage caused by offloading duplicate tasks, thereby optimizing resource utilization. In order to improve the overall system utility, Yan et al.³⁷ studied the task offloading problem in unmanned surface vehicles network and proposed an algorithm based on deep reinforcement learning for optimizing load balancing in task offloading. Their proposed algorithm enables the selection of the most suitable edge server or cloud server for efficient offloading, thereby improving the overall system performance.

Lu et al.³⁸ proposed a task offloading strategy based on deep reinforcement learning, which improves the deep Q-network algorithm by incorporating an long short-term memory network layer and a candidate network set. The objective of their strategy is to optimize latency, energy consumption, and load balancing. Hossain et al.³⁹ proposed a task offloading management scheme based on fuzzy decision-making, which takes into account both vertical and horizontal task offloading scenarios to cater to diverse user requirements. Their scheme prioritizes the balancing of network and computational resources when making offloading decisions, ensuring optimal utilization of available resources.

Task offloading with joint security

Nguyen et al.⁴⁰ formulated the joint optimization problem of task offloading and privacy preservation as a Markov decision process model. They proposed a reinforcement learning-based task offloading algorithm specifically designed for a dynamic blockchain network. Samy et al.⁴¹ propose a framework based on the blockchain technology for secure task offloading, ensuring guaranteed performance in terms of execution latency and energy consumption. They formulated the joint task offloading model as an NP-hard model and proposed a deep reinforcement learning-based algorithm to address it. Xu et al.⁴² proposed a task offloading method for video surveillance in the Internet of vehicles, aiming to reduce time costs, maintain load balancing of edge nodes, and enhance privacy protection. They formulated the joint problem as a multi-objective optimization model and employed the technique for order preference by similarity to ideal solution (TOPSIS) to solve it. Wei et al.⁴³ investigated the problem of personalized privacy-aware task offloading in the industrial IoT. They proposed an approach based on local differential privacy and deep reinforcement learning to address this challenge. Lingayya et al.⁴⁴ introduced a privacy-preserving framework based on Petri nets, encompassing techniques such as synchronization, sequential execution, concurrency, and conflict resolution. They employed a multi-agent-based cohesion approach to ensure consistent protection of user privacy and secure services in the face of the challenges posed by diverse tasks and global data management. Dai et al.⁴⁵ proposed a learning-based vehicle-to-vehicle offloading algorithm, which achieves low-latency and high-efficiency task offloading in dynamic vehicular networks by comprehensively evaluating the computational capacity and privacy risk of server vehicles.

Brief summary

The task offloading problem is commonly formulated as an optimization model, aiming to optimize various indicators such as latency, cost, energy consumption, and other relevant factors. The basic task offloading focuses solely on making reasonable offloading decisions to optimize EC indicators within, without incorporating any supplementary mechanisms^{22–32}. In scenarios involving task offloading across multiple edge servers, the need for load balancing becomes evident and imperative. By balancing the workload across the available edge servers, the system can avoid resource bottlenecks, and prevent overloading of specific servers^{33–39}. Although EC holds strong potentiality in terms of security, unreasonable task offloading will undermine this potentiality, leading to the risk of task information leakage^{46,47}. Therefore, it is imperative to implement requisite measures to ensure the security of EC⁴⁸. Security concerns associated with task offloading have garnered significant attention, compelling researchers to actively address and incorporate security protection measures within their offloading solutions^{40–45}.

Comparison of the related work is illustrated in Table 1. The existing work on basic task offloading predominantly concentrates on establishing optimization models for different scenarios and optimizing some indicators, often neglecting the incorporation of supplementary mechanisms to further enhance performance and address specific challenges. The basic task offloading lacks the utilization of collaborative capabilities among multiple edge servers, fails to adapt to the latency and security requirements of industrial Internet, and does not fully harness the maximum potentiality offered by EC. While some studies have incorporated load balancing into task offloading, their focus primarily lies within civil scenarios, lacking in-depth exploration of the key aspects pertaining to the industrial Internet. Existing task offloading approaches that incorporate security protections primarily rely on encryption algorithms and blockchain technology, which introduce additional overhead and potentially hinder the efficiency of EC task offloading.

Different from existing work, this paper focuses on the problem of latency-aware multi-server EC partial task offloading in the industrial Internet. We integrate load balancing and fuzzy security protection mechanisms to optimize the utilization of edge server resources and provide a more precise characterization of task risks. Furthermore, we leverage a partial offloading mode to facilitate task offloading at a fine-grained level. To address the composite offloading strategy derived from the established model, we propose a bi-layer offloading algorithm with joint load balance and fuzzy security, which is based on the adaptive genetic algorithm and simulated annealing particle swarm optimization. In contrast to existing work, our proposed approach offers the support for latency-aware multi-server EC partial task offloading in the industrial Internet with joint load balancing and fuzzy security, which has not been extensively explored before.

System model and problem formulation

Scenario description

The system architecture, as depicted in Fig. 1, comprises a collection of industrial terminals and a set of edge servers. Let N represent the number of industrial terminals, and M denote the number of edge servers in the system. Due to their primary responsibility in manufacturing operations and the constraints imposed by the manufacturing environment, industrial terminals exhibit a significantly limited resource capacity in terms of computing power. Under the system architecture illustrated in Fig. 1, tasks on the industrial terminals can be partially offloaded to edge servers via wireless networks, thereby expanding the available computing resources of the industrial terminals. The system comprises multiple edge servers available for selection by industrial

Work	Scenario	Offloading mode	Load balance	Security protection	Objective
22	Industrial IoT	Binary offloading	×	×	Minimize the latency, energy consumption and task execution cost
23	Healthcare IoT	Binary offloading	×	×	Minimize the long-term energy consumption under latency requirements
24	Industrial IoT	Binary offloading	×	×	Optimize the tradeoff of latency and energy consumption
25	Industrial IoT	Partial offloading	×	×	Minimize the latency
26	Ultra-dense network	Binary offloading	×	×	Minimize the latency and energy consumption
27	Industrial IoT	Binary offloading	×	×	Minimize the latency and offloading cost
28	Industrial Internet	Binary offloading	×	×	Minimize the system overhead
29	Cell network	Partial offloading	×	×	Minimize the latency
33	Cell network	Partial offloading	×	×	Minimize the latency
34	Ultra-dense network	Partial offloading	✓	×	Minimize the load
35	IoT	Partial offloading	✓	×	Minimize the make-span
36	Internet of vehicles	Binary offloading	✓	×	Minimize the latency
37	Surface unmanned boat network	Binary offloading	✓	×	Minimize the latency and energy consumption
38	Cell network	Partial offloading	✓	×	Optimize the latency, energy consumption, and load balancing
39	Cell network	Binary offloading	✓	×	Minimize the latency and failure rate
40	Cell network	Binary offloading	×	✓	Minimize the latency and energy consumption while guaranteeing the privacy
41	Cell network	Binary offloading	×	✓	Minimize the latency and energy consumption
42	Internet of vehicles	Binary offloading	✓	✓	Optimize the latency, load balancing, and privacy entropy
43	Industrial IoT	Partial offloading	×	✓	Maximize the reward
44	IoT	Binary offloading	×	✓	Minimize the energy consumption
30	Internet of vehicles	Binary offloading	×	×	Maximize the profit
31	Space-air-ground network	Binary offloading	×	×	Maximize the profit
32	Vehicles-to-vehicles	Binary offloading	×	×	Maximize the overall network throughput
45	Vehicles-to-vehicles	Binary offloading	✓	✓	Minimize the latency
Ours	Industrial Internet	Partial offloading	✓	✓	Balance the load, minimize the latency and security risks

Table 1. Comparison of the related work.

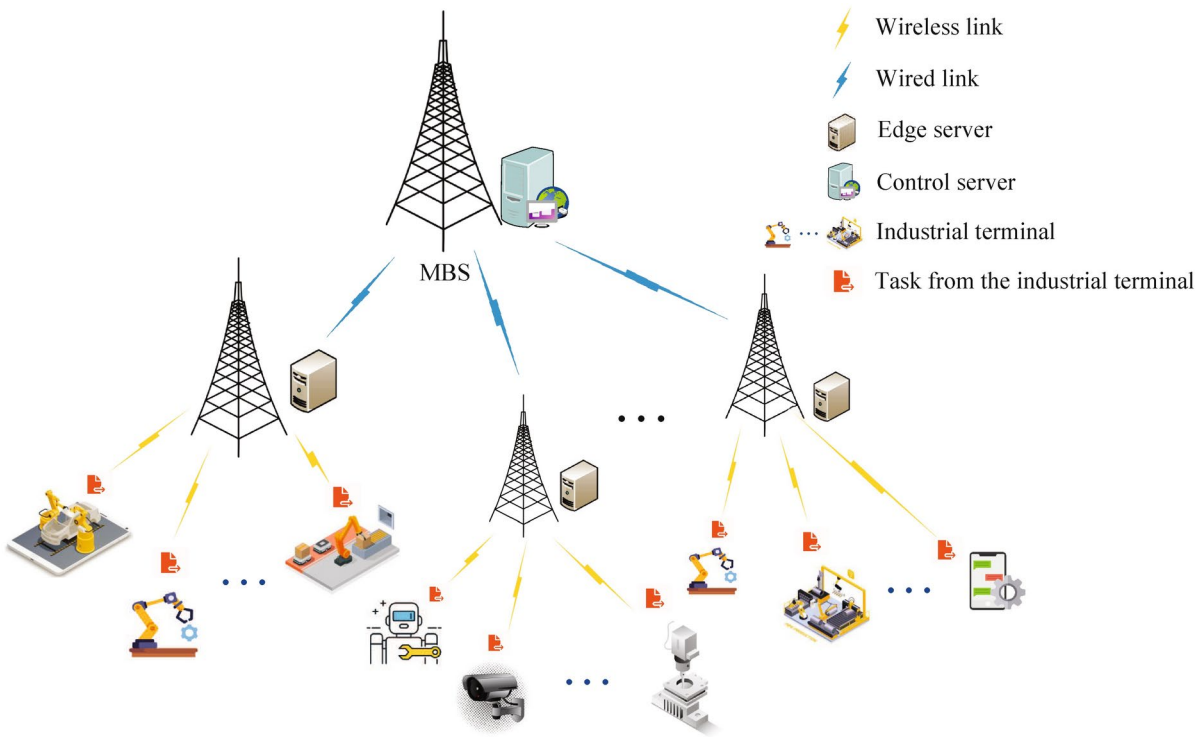


Fig. 1. The system architecture.

computing tasks. When offloading tasks from industrial terminals to edge servers, decisions need to be made regarding which server to offload to and the proportion of tasks to be offloaded for execution on the destination edge server. Additionally, it is necessary to consider the load balancing across multiple servers, ensuring that task execution meets the requirements of low latency and security. The task offloading decisions are orchestrated by the control server situated at the macro base station (MBS). The notations used in this paper are summarized in Table 2.

System model

Computation model

Let $H_i = (D_i, C_i, \hat{S}_i)$ denote the computing task on the i th ($1 \leq i \leq N$) industrial terminal. D_i represents its data size, C_i (in CPU cycles per bit) represents the number of CPU cycles required to compute one bit of data, and $\hat{S}_i$ represents its fuzzy security level. The offloading strategy for task H_i is composed of two distinct components. One component is the offloading ratio, denoted as a continuous variable x_i ($x_i \in [0, 1]$). x_i denotes the proportion of task H_i that is offloaded to the edge server for execution. As the value of x_i increases, a greater amount of computing overload will be offloaded to the edge server. Another component is the offloading location, represented by a binary variable y_{ij} ($y_{ij} \in \{0, 1\}$). y_{ij} indicates whether task H_i is offloaded to the j th

Notation	Definition
H_i	The computing task on the i th industrial terminal
E_j	The j th edge server
N	The number of industrial terminals
M	The number of edge servers
D_i	The data size of task H_i
C_i	The number of CPU cycles required to compute one bit data of task H_i
$\hat{S}_i$	The fuzzy security level of task H_i
$\mathbf{X}$	The offloading ratio strategy, whose element is x_i
$\mathbf{Y}$	The offloading location strategy, whose element is y_{ij}
f_i^l	The CPU frequency of the i th industrial terminal
f_j^e	The CPU frequency allocated from edge server E_j
r_{ij}	The wireless network rate between the i th industrial terminal and edge server E_j
b_i	The wireless bandwidth allocated to the i th industrial terminal
p_{ij}	The transmission power of the i th industrial terminal when transmitting data to edge server E_j
g_i	The gain of the channel used to transmit data from the i th industrial terminal
ζ	The noise power
$\alpha_1 \sim \alpha_3$	Weights used to calculate the load level
Ψ_j, Ω_j	The CPU and memory capacities of edge server E_j
Λ	The wireless bandwidth capacity
L	The number of factory experts who evaluate the task risk
ε	The weight matrix of factory experts' evaluation in the overall task risk
u_k	The k th viewpoint for measuring the task security level
η_k	The weight of u_k when calculating the security level of task H_i
μ_i	The security risk of task H_i
θ	The security threshold
$\delta_1 \sim \delta_3$	Weights of latency, security risk, and load variance in the objective function
P_i	The probability of the i th individual being selected in the upper-layer algorithm
P_c, P_m	The crossover and mutation probabilities in the upper-layer algorithm
$\mathbf{X}^{bp}[j]$	The individual best position of the j th particle in the lower-layer algorithm
$\mathbf{X}(iter)[j]$	The position of the j th particle in lower-layer algorithm's $iter$ th iteration
$\mathbf{X}^b$	The global best position in the lower-layer algorithm
$\mathbf{X}^b(iter)$	The global best position in lower-layer algorithm's $iter$ th iteration
$\mathbf{V}(iter)[j]$	The velocity of the j th particle in lower-layer algorithm's $iter$ th iteration
$\chi(iter)$	The inertia factor in lower-layer algorithm's $iter$ th iteration
ϑ_1, ϑ_2	The random factors of the lower-layer algorithm
c_1, c_2	The learning factors of the lower-layer algorithm

Table 2. Notations.

($1 \leq j \leq M$) edge server E_j or not. $y_{ij} = 1$ means that task H_i is offloaded to edge server E_j , while $y_{ij} = 0$ means that task H_i is not offloaded to edge server E_j .

Although the computing power of industrial terminals is limited, they also have the possibility to perform a certain proportion of tasks. Let f_i^l denote the CPU frequency of the i th industrial terminal. When a fraction of the task is executed locally at the i th industrial terminal, the computation latency $t_i^l(x_i)$ is expressed as

$$t_i^l(x_i) = \frac{C_i^l(x_i)}{f_i^l}, \quad (1)$$

in which $C_i^l(x_i) = (1 - x_i)D_iC_i$.

Let f_j^e denote the CPU frequency allocated from edge server E_j . When a portion of the task is offloaded to edge server E_j for execution, the computation latency $t_{ij}^e(x_i, y_{ij})$ is expressed as

$$t_{ij}^e(x_i, y_{ij}) = \frac{\beta_{ij}(x_i, y_{ij})}{f_j^e}, \quad (2)$$

in which $\beta_{ij}(x_i, y_{ij}) = x_i y_{ij} D_i C_i$.

Communication model

During the process of task offloading to edge servers, wireless networks are utilized for transmitting data between industrial terminals and edge servers. However, this data transmission introduces communication latency, which should be counted into the overall latency of task offloading in addition to the computation latency. The presence of additional communication latency significantly influences the efficacy of task offloading. Excessive communication latency can potentially negate the advantages of task offloading and may even yield adverse outcomes, leading to inferior task offloading performance compared to local execution on industrial terminals.

The transmission latency required to offload task H_i to edge server E_j is

$$t_{ij}^{tr}(x_i, y_{ij}) = \frac{x_i y_{ij} D_i}{r_{ij}}, \quad (3)$$

in which r_{ij} represents the wireless network rate between the i th industrial terminal and edge server E_j . According to Shannon's theory⁴⁹, r_{ij} is expressed as

$$r_{ij} = b_i \log_2 \left(1 + \frac{p_{ij} g_i^2}{b_i \zeta} \right), \quad (4)$$

in which b_i represents the wireless bandwidth allocated to the i th industrial terminal, p_{ij} represents the transmission power of the i th industrial terminal when transmitting data to edge server E_j , g_i represents the gain of the channel used to transmit data from the i th industrial terminal, and ζ represents the noise power.

Load balance model

In the endeavor of load balance, CPU, bandwidth, and memory are chosen as the target resources to be balanced, as they serve as pivotal factors influencing system performance and task execution⁵⁰. By considering these three resources, we aim to optimize resource utilization, elevate overall system efficiency, and ensure effective task offloading and execution. We define the load level of edge server E_j as

$$\Gamma_j(y_{ij}) = \alpha_1 \psi_j(y_{ij}) + \alpha_2 \lambda_j(y_{ij}) + \alpha_3 \omega_j(y_{ij}), \quad (5)$$

in which $\psi_j(y_{ij})$, $\lambda_j(y_{ij})$ and $\omega_j(y_{ij})$ represent edge server E_j 's resource utilization of CPU, bandwidth, and memory, respectively. $\alpha_1 \sim \alpha_3$ denote their weights and are constrained by $\alpha_1 + \alpha_2 + \alpha_3 = 1$. $\psi_j(y_{ij})$, $\lambda_j(y_{ij})$ and $\omega_j(y_{ij})$ are expressed as follows, with the constraint that their values do not exceed 1.

$$\psi_j(y_{ij}) = \frac{\sum_{i=1}^N y_{ij} f_j^e}{\Psi_j} \leq 1 \quad (6)$$

$$\lambda_j(y_{ij}) = \frac{\sum_{i=1}^N y_{ij} b_i}{\Lambda} \leq 1 \quad (7)$$

$$\omega_j(y_{ij}) = \frac{\sum_{i=1}^N y_{ij} m_i}{\Omega_j} \leq 1 \quad (8)$$

f_j^e represents the allocated CPU resources from edge server E_j , b_i represents the wireless bandwidth allocated to the i th industrial terminal, and m_i represents the memory allocated to the i th industrial terminal. Ψ_j and Ω_j represent the CPU capacity and memory capacity of edge server E_j , respectively. Λ represents the wireless bandwidth capacity.

Let $\bar{\Gamma}_j(y_{ij})$ denote the average load level of M edge servers, expressed as

$$\bar{\Gamma}_j(y_{ij}) = \frac{\sum_{j=1}^M \Gamma_j(y_{ij})}{M}. \quad (9)$$

Load variance is a measure of the variability of load distribution among different edge servers through the concept of variance. When the load is unevenly distributed, the variance value of edge server load will be more significant, reflecting the task offload imbalance. This imbalance may cause some edge servers to be overloaded while others are lightly loaded, thus affecting the performance and stability of the entire system. Let $\Phi(y_{ij})$ denote the load variance, expressed as

$$\Phi(y_{ij}) = \frac{1}{M} \sum_{j=1}^M (\Gamma_j(y_{ij}) - \bar{\Gamma}_j(y_{ij}))^2. \quad (10)$$

Fuzzy security model

The triangular fuzzy number is utilized to effectively characterize the security level associated with the industrial task. To assess the security level of a task comprehensively, it is necessary to consider multiple perspectives. In this paper, we employ set $\{u_1, u_2, u_3\} = \{\text{Task data, Task type, Terminal lifecycle}\}$ to represent the viewpoint for measuring task security level. To facilitate the comprehensive evaluation of the security level, we employ an evaluation set $\{\text{High, Medium, Low}\}$, whose assessment scoring levels are shown in Table 3.

For example, from the perspective of task type (u_2), a triangular fuzzy number (0.28, 0.65, 0.86) can be assigned by a factory expert to represent the security level of task H_i . In this example, the value 0.28 represents the lower bound of the security level assessment, indicating the expert's minimum evaluation within the given task type. The value 0.65 represents the expert's most probable assessment considering the task type. Lastly, the value 0.86 denotes the upper bound of the security level assessment, reflecting the expert's highest evaluation based on the given task type.

We assume that L factory experts evaluate the task security level from different perspectives (u_1, u_2 , and u_3). Let $\hat{S}_i^{(k)} = (\underline{s}_i^{(k)}, s_i^{(k)}, \bar{s}_i^{(k)})$ denote the task security level assessment result given by the l th expert from the perspective of u_k . Since experts have different working experiences, a weight matrix $\varepsilon = [\varepsilon_l]_L$ is set for each expert's evaluation in the overall evaluation. The evaluation result of the security level of N tasks from the perspective of u_k by L experts is expressed

$$\begin{aligned} \hat{S}^{(k)} &= [\varepsilon_1, \varepsilon_2, \dots, \varepsilon_L] \begin{bmatrix} \underline{s}_{11}^{(k)}, s_{11}^{(k)}, \bar{s}_{11}^{(k)} & \underline{s}_{12}^{(k)}, s_{12}^{(k)}, \bar{s}_{12}^{(k)} & \dots & \underline{s}_{1N}^{(k)}, s_{1N}^{(k)}, \bar{s}_{1N}^{(k)} \\ \underline{s}_{21}^{(k)}, s_{21}^{(k)}, \bar{s}_{21}^{(k)} & \underline{s}_{22}^{(k)}, s_{22}^{(k)}, \bar{s}_{22}^{(k)} & \dots & \underline{s}_{2N}^{(k)}, s_{2N}^{(k)}, \bar{s}_{2N}^{(k)} \\ \dots & \dots & \dots & \dots \\ \underline{s}_{L1}^{(k)}, s_{L1}^{(k)}, \bar{s}_{L1}^{(k)} & \underline{s}_{L2}^{(k)}, s_{L2}^{(k)}, \bar{s}_{L2}^{(k)} & \dots & \underline{s}_{LN}^{(k)}, s_{LN}^{(k)}, \bar{s}_{LN}^{(k)} \end{bmatrix} \\ &= \left[\underline{s}_1^{(k)}, s_1^{(k)}, \bar{s}_1^{(k)}, \underline{s}_2^{(k)}, s_2^{(k)}, \bar{s}_2^{(k)}, \dots, \underline{s}_N^{(k)}, s_N^{(k)}, \bar{s}_N^{(k)} \right], \end{aligned} \quad (11)$$

in which $\hat{S}_i^{(k)} = (\underline{s}_i^{(k)}, s_i^{(k)}, \bar{s}_i^{(k)})$ represents task H_i 's fuzzy security level from the perspective of u_k . Correspondingly, security level $\hat{S}_i$ of task H_i can be expressed as $\hat{S}_i = [\hat{S}_i^{(k)}]_{k=1}^3$. $\underline{s}_i^{(k)} = \sum_{l=1}^L \varepsilon_l \underline{s}_{li}^{(k)}$ represent the lower bound of the weighted evaluation result for H_i from the perspective of u_k . Similarly, $s_i^{(k)} = \sum_{l=1}^L \varepsilon_l s_{li}^{(k)}$ denotes the most possible value of the weighted evaluation result for H_i from u_k , while $\bar{s}_i^{(k)} = \sum_{l=1}^L \varepsilon_l \bar{s}_{li}^{(k)}$ signifies the upper bound of the weighted evaluation result from u_k .

The fuzzy transformation method is a viable approach for converting a triangular fuzzy number into a single value. Let $S^{(k)} = [S_1^{(k)}, \dots, S_i^{(k)}, \dots, S_N^{(k)}]$ represent the converted matrix from $\hat{S}^{(k)} = [\underline{s}_1^{(k)}, s_1^{(k)}, \bar{s}_1^{(k)}, \underline{s}_2^{(k)}, s_2^{(k)}, \bar{s}_2^{(k)}, \dots, \underline{s}_N^{(k)}, s_N^{(k)}, \bar{s}_N^{(k)}]$, where each element is determined using the transformation method proposed by Dong et al.⁵¹ The transformed element, denoted as $S_i^{(k)}$, is expressed as

$$S_i^{(k)} = \frac{\underline{s}_i^{(k)} + 4s_i^{(k)} + \bar{s}_i^{(k)}}{6}. \quad (12)$$

Low	Medium	High
[0, 0.4]	[0.4, 0.7]	[0.7, 1]

Table 3. The assessment scoring level.

Considering that different u_k may have varying degrees of influence, distinct weights are employed to account for their respective impacts. Let η_k denote the weight of u_k , where $\eta_k \geq 0$, $\sum_{k=1}^3 \eta_k = 1$. In summary, the security level of N tasks is expressed as

$$\begin{aligned} \mathbf{S} &= [\eta_1, \eta_2, \eta_3][\mathbf{S}^{(1)}, \mathbf{S}^{(2)}, \mathbf{S}^{(3)}] \\ &= [\eta_1, \eta_2, \eta_3] \begin{bmatrix} S_1^{(1)} & S_2^{(1)} & \dots & S_N^{(1)} \\ S_1^{(2)} & S_2^{(2)} & \dots & S_N^{(2)} \\ S_1^{(3)} & S_2^{(3)} & \dots & S_N^{(3)} \end{bmatrix} \\ &= [\eta_1 S_1^{(1)} + \eta_2 S_1^{(2)} + \eta_3 S_1^{(3)}, \dots, \eta_1 S_N^{(1)} + \eta_2 S_N^{(2)} + \eta_3 S_N^{(3)}], \end{aligned} \quad (13)$$

whose element $S_i = \eta_1 S_i^{(1)} + \eta_2 S_i^{(2)} + \eta_3 S_i^{(3)}$ represents task H_i 's security level.

We use security risks to quantitatively evaluate the security degree during task offloading process. The security risk of task H_i is expressed as

$$\mu_i = \begin{cases} 1 - e^{-(\theta - S_i)}, & \theta > S_i \\ 0, & \text{otherwise} \end{cases}, \quad (14)$$

in which θ represents the security threshold. In the case where the task security level exceeds the predetermined threshold, a security risk is present. An increasing trend in security risks is associated with a decreasing task security level. Conversely, when the security level is below or equal to the threshold, the security risk is assigned a value of 0. The security risk of task H_i in the offloading process is expressed as

$$\xi_{ij}(x_i, y_{ij}) = x_i y_{ij} \mu_i. \quad (15)$$

Problem formulation

The problem formulation provides a mathematical description of the latency-aware partial task offloading problem for multi-server EC in the industrial Internet with joint load balancing and security protection. This formulation lays the foundation for quantitatively addressing the optimal composite offloading strategy. This paper strives to achieve three key objectives: minimizing latency, mitigating security risks, and reducing load variance.

The latency incurred during task offloading encompasses the transmission latency and the computation latency introduced by the edge server, in addition to the inherent latency associated with local computation latency. The offloading latency of task H_i is calculated by

$$T_i(x_i, y_{ij}) = t_i^l(x_i) + t_{ij}^{tr}(x_i, y_{ij}) + t_{ij}^e(x_i, y_{ij}), \quad (16)$$

in which $t_i^l(x_i)$ represents task H_i 's local computation latency, $t_{ij}^{tr}(x_i, y_{ij})$ represents the transmission latency of task when it is offloaded to edge server E_j , and $t_{ij}^e(x_i, y_{ij})$ represents the computation latency of task when it is offloaded to edge server E_j . The total execution latency of N tasks is expressed as

$$T^{total}(\mathbf{X}, \mathbf{Y}) = \sum_{i=1}^N T_i(x_i, y_{ij}), \quad (17)$$

in which $\mathbf{X} = [x_i]_N$, and $\mathbf{Y} = [y_{ij}]_{N \times M}$. The normalized latency $T(\mathbf{X}, \mathbf{Y})$ is calculated by $T(\mathbf{X}, \mathbf{Y}) = T^{total}(\mathbf{X}, \mathbf{Y})/T_{\max}$, in which $T_{\max}$ represents the maximum latency. The total security risk of N tasks is expressed as

$$\xi^{total}(\mathbf{X}, \mathbf{Y}) = \sum_{i=1}^N \xi(x_i, y_{ij}). \quad (18)$$

The normalized security risk $\xi(\mathbf{X}, \mathbf{Y})$ is calculated by $\xi(\mathbf{X}, \mathbf{Y}) = \xi^{total}(\mathbf{X}, \mathbf{Y})/\xi_{\max}$, in which $\xi_{\max}$ represents the maximum security risk. Since the load variance is calculated based on resource utilization, which is already normalized, we integrate these three optimization objectives into a single objective function using the linear weighting method. The objective function $F(\mathbf{X}, \mathbf{Y})$ is expressed as

$$F(\mathbf{X}, \mathbf{Y}) = \delta_1 T(\mathbf{X}, \mathbf{Y}) + \delta_2 \xi(\mathbf{X}, \mathbf{Y}) + \delta_3 \Phi(\mathbf{Y}), \quad (19)$$

in which $\delta_1 \sim \delta_3$ represent the weights of latency, security risk, and load variance.

Our objective is to simultaneously optimize latency, security risk, and load variance by adjusting the composite offloading strategy composed of offloading location and offloading ratio. The task offloading problem is formulated as objective function (20) with constraints (6–8, 21–23):

$$\min F(\mathbf{X}, \mathbf{Y}) \quad (20)$$

s.t.

$$(6), (7), (8)$$

$$0 \leq x_i \leq 1 \quad (21)$$

$$y_{ij} \in \{0, 1\} \quad (22)$$

$$\delta_1 + \delta_2 + \delta_3 = 1 \quad (23)$$

Formula (20) represents the minimization of the objective function, in which the offloading ratio X and the offloading location Y are independent variables, while the objective function F , composed of latency, security risk, and load variance, serves as the dependent variable. Constraint (6) guarantees that the CPU capacity of edge server E_j remains unexceeded. Constraint (7) ensures that the bandwidth of edge server E_j remains within the specified limit. Constraint (8) ensures that the memory capacity of edge server E_j will not be surpassed. Constraint (21) ensures that the offloading ratio x_i for task H_i remains a continuous value between 0 and 1. Constraint (22) enforces that the offloading location y_{ij} can only be binary values of 0 or 1. Constraint (23) guarantees that the summation of the three weights equals 1.

Solution space analysis of the system model

The offloading problem is formulated as a constrained combinatorial optimization model, as shown in formulas (6–8, 20–23). This model is hybrid in nature, incorporating both offloading ratios X and offloading locations Y . In terms of offloading locations, for each task, there are $M + 1$ possible locations (including M edge servers and the option for local execution). Therefore, for N tasks, the total number of possible solutions is $(M + 1)^N$. As a result, the solution space of the optimization model is $(M + 1)^N$, indicating that the solution space grows exponentially with the number of tasks and increases idempotently with the number of edge servers. The complexity of the system model is quite high, posing significant challenges in solving it.

As the number of tasks and edge servers increases, the solution space expands rapidly, leading to a combinatorial explosion that must be accounted for. Although the idempotent increase in the number of edge servers is less dramatic than the exponential growth in the number of tasks, it still adds significant computational burden to the task offloading optimization process. This rapid expansion leads to substantial computational challenges, making exhaustive searches of the entire solution space infeasible in most practical scenarios. Therefore, as the solution space grows, it becomes critical to employ efficient offloading algorithms that balance solution quality with computational efficiency.

NP-hardness proof of the system model

Theorem 1 *The established task offloading optimization model is NP-hard.*

Proof The knapsack model is a classical combinatorial optimization model, in which a subset of items is chosen from a given set, with the objective of optimizing the objective value while respecting the capacity constraint of the knapsack. The knapsack model has been proven to be an NP-hard model⁵², indicating that it cannot be solved in polynomial time. Specifically, the 0-1 knapsack problem is a variant of the knapsack problem where each item is restricted to either being selected or not selected for inclusion in the knapsack. The knapsack has a fixed capacity, and the total weight of the chosen items must not exceed this capacity limit. Suppose there is a set of N items, where the i th ($1 \leq i \leq N$) item has the corresponding value v_i and weight w_i . The mathematical formulation of the backpack model can be described as follows:

$$\max \quad f(Z) \quad (24)$$

s.t.

$$z_i \in \{0, 1\} \quad (25)$$

$$\sum_{i=1}^n w_i z_i \leq W \quad (26)$$

$f(Z) = \sum_{i=1}^N v_i z_i$ represents the objective function. z_i is the element of $Z = [z_i]_N$ and indicates whether the i th item is selected to be placed in the backpack or not. Constraint (25) ensures that the value of z_i is binary. The binary variable z_i is assigned a value of 1 to indicate the selection of the i th item for placement in the knapsack, while a value of 0 signifies that the i th item is not chosen for inclusion in the knapsack. Constraint (26) guarantees that the total weight of the selected items does not exceed the capacity limit W of the knapsack.

We conduct an analysis on a simplified version of our task offloading optimization model, focusing solely on latency, a single edge server, and binary offloading mode. In the simplified task offloading optimization model, the objective is to efficiently offload suitable tasks to the edge server with the aim of minimizing latency.

By regarding N tasks as N items and perceiving the edge server as the knapsack, the simplified model can be analogized to a knapsack scenario, where the objective entails selecting (offloading) the appropriate items (tasks) for inclusion within the knapsack (edge server). Since the simplified model has only one edge server, the number of edge servers $M = 1$, and the offloading location strategy degenerates from $\mathbf{Y} = [y_{ij}]_{N \times M}$ to $\mathbf{Y} = [y_i]_N$. Simultaneously, due to the simplification of partial offloading to binary offloading within the simplified model, the offloading ratio strategy x_i and the offloading location strategy y_i become equivalent. The simplified objective function can be expressed as

$$\begin{aligned} F(\mathbf{Y}) &= \sum_{i=1}^N t_i^l(y_i) + t_{i1}^{tr}(y_i) + t_{i1}^e(y_i) \\ &= \sum_{i=1}^N \frac{(1-y_i)D_iC_i}{f_i^l} + \frac{D_iy_i}{r_{i1}} + \frac{D_iC_iy_i}{f_1^e} \\ &= \sum_{i=1}^N \frac{D_iC_i}{f_i^l} - \sum_{i=1}^N \left(\frac{D_iC_i}{f_i^l} - \frac{D_i}{r_{i1}} - \frac{D_iC_i}{f_1^e} \right) y_i. \end{aligned} \quad (27)$$

Define an objective function $t(\mathbf{Y})$ as

$$t(\mathbf{Y}) = \sum_{i=1}^N \left(\frac{D_iC_i}{f_i^l} - \frac{D_i}{r_{i1}} - \frac{D_iC_i}{f_1^e} \right) y_i. \quad (28)$$

Let t_i^e denote the computation latency when task H_i is executed on the edge server. Let T^e denote the computation latency of executing N tasks on the edge server. Then, the simplified task offloading optimization model can be formulated as

$$\max \quad t(\mathbf{Y}) \quad (29)$$

s.t.

$$y_i \in \{0, 1\} \quad (30)$$

$$\sum_{i=1}^N t_i^e y_i \leq T^e \quad (31)$$

Formula (29) presents the objective function formulated to minimize the latency. Constraint (30) imposes binary restrictions on the variables, ensuring that their values can only be 0 or 1. Constraint (31) ensures that the latency resulting from offloaded tasks executed on the edge server does not exceed the overall latency incurred by executing all tasks solely on the edge server. Formulas (29)–(31) mathematically convert the simplified task offloading optimization model into the knapsack model. Our complete task offloading optimization model extends the simplified model by incorporating multiple edge servers, partial offloading mode, load balancing, and fuzzy security. This extension significantly increases the complexity compared to the simplified model, which is mathematically transformed into the NP-hard knapsack model presented in formulas (29)–(31). Henceforth, the established task offloading optimization model is demonstrated to be NP-hard. $\square$

Description of the proposed offloading algorithm

Overview

Accurately solving NP-hard problems within the polynomial time is infeasible, as algorithms designed to solve such problems often exhibit exponential time complexity. Evolutionary algorithms, such as genetic algorithm and particle swarm optimization, offer efficient alternatives to exhaustive search by exploring the solution space in an adaptive manner. These algorithms can provide near-optimal solutions within a reasonable time, which makes them suitable for addressing the computational challenges of task offloading in multi-server EC environments. In this paper, we propose a bi-layer offloading algorithm with joint load balance and fuzzy security, which is based on the adaptive genetic algorithm and simulated annealing particle swarm optimization, to solve the established NP-hard model.

The genetic algorithm is a widely-used evolutionary algorithm inspired by the process of natural selection. It is particularly effective in solving discrete optimization problems by simulating evolution through operators such as selection, crossover, and mutation. In task offloading, it optimizes the offloading location by assigning tasks to the most appropriate edge server based on resources and network conditions⁵³. On the other hand, particle swarm optimization is well-suited for continuous optimization problems, such as the offloading ratio decision. It mimics the social behavior of birds flocking, where each particle adjusts its position based on its own experience and that of its neighbors, ultimately converging to an optimal or near-optimal solution²². These methods are foundational in our proposed bi-layer offloading algorithm, with the genetic algorithm handling the discrete assignment of tasks to edge servers and particle swarm optimization determining the offloading ratio.

Given the inherent coupling between the offloading ratio decision and the offloading location decision within the established optimization model, our approach decomposes the offloading problem into bi-layer to effectively address this interdependence. The upper layer is responsible for determining the task's offload location (i.e., which edge server will handle the task), while the lower layer focuses on optimizing the offloading ratio (i.e., the proportion of the task to be offloaded). The upper layer uses a discrete decision variable y_{ij} , representing the assignment of the offloaded task to a specific edge server. In contrast, the lower layer uses a continuous variable x_{ij} , representing the proportion of task i to be offloaded to server j once the location is determined.

The proposed offloading algorithm employs adaptive genetic algorithm and simulated annealing particle swarm optimization in the upper and lower layers, respectively. The key idea revolves around utilizing an adaptive genetic algorithm to address the discrete offloading location strategy within the upper layer, while employing a simulated annealing particle swarm algorithm to tackle the continuous offloading ratio strategy within the lower layer. The optimized offloading ratio from the lower-layer algorithm is fed back to the upper-layer algorithm to refine subsequent task allocation decisions. The pseudo-codes for the upper-layer and lower-layer algorithms are presented in Algorithms 1 and 2, respectively.

Upper-layer algorithm based on the adaptive genetic algorithm

Input: Size of the genetic population NP_1 ; Size of the particle swarm population NP_2 ; Number of upper-layer iterations $Iter_1$; Number of lower-layer iterations $Iter_2$; N tasks; M edge servers; Weight factors $\delta_1, \delta_2, \delta_3$.

Output: Optimal task offloading ratio $\mathbf{X}^b$; Optimal task offloading location $\mathbf{Y}^b$.

```

1: Generate the initial population  $\mathbf{Y}(0)[NP_1]$  randomly;
2: The lower-layer algorithm Algorithm 2 is performed repeatedly to obtain the optimal  $\mathbf{X}^b(0)[NP_1]$  for  $\mathbf{Y}(0)[NP_1]$ ;
3: Find the best  $\mathbf{Y}^b$  from  $\mathbf{Y}(0)[NP_1]$  and its corresponding best  $\mathbf{X}^b$  from  $\mathbf{X}^b(0)[NP_1]$ ;
4: for  $iter = 1$  to  $Iter_1$  do
5:   for  $i = 1$  to  $NP_1/2$  do
6:     Select two individuals  $\mathbf{Y}_1$  and  $\mathbf{Y}_2$  from  $\mathbf{Y}(iter-1)[NP_1]$  based on the probability calculated by Equation (33);
7:     Calculate the adaptive crossover probability  $P_c$  by Equation (34);
8:     if  $random(0, 1) < P_c$  then
9:       Perform the crossover operation on individuals  $\mathbf{Y}_1$  and  $\mathbf{Y}_2$ ;
10:    end if
11:    for  $j = 1$  to 2 do
12:      Calculate the adaptive mutation probability  $P_m$  by Equation (35);
13:      if  $random(0, 1) < P_m$  then
14:        Perform the mutation operation on individual  $\mathbf{Y}_j$ ;
15:      end if
16:    end for
17:    The lower-layer algorithm Algorithm 2 is performed to obtain the optimal  $\mathbf{X}_1^b, \mathbf{X}_2^b$  for  $\mathbf{Y}_1, \mathbf{Y}_2$ ;
18:    Save  $\mathbf{Y}_1$  and  $\mathbf{Y}_2$  to  $\mathbf{Y}(iter)[NP_1]$ ;
19:    Save  $\mathbf{X}_1^b$  and  $\mathbf{X}_2^b$  to  $\mathbf{X}^b(iter)[NP_1]$ ;
20:  end for
21:  Find the best individual  $\mathbf{Y}^b(iter)$  from  $\mathbf{Y}(iter)[NP_1]$  and its corresponding best  $\mathbf{X}^b(iter)$  from  $\mathbf{X}^b(iter)[NP_1]$ ;
22:  if  $f_u(\mathbf{X}^b(iter), \mathbf{Y}^b(iter)) > f_u(\mathbf{X}^b, \mathbf{Y}^b)$  then
23:     $\mathbf{X}^b \leftarrow \mathbf{X}^b(iter)$ ;
24:     $\mathbf{Y}^b \leftarrow \mathbf{Y}^b(iter)$ ;
25:  end if
26: end for
27: return  $\mathbf{X}^a$  and  $\mathbf{Y}^a$ 

```

Algorithm 1. The upper-layer algorithm

Initialization

Initialization plays a crucial role in the upper-layer algorithm as it sets the foundation for the subsequent evolutionary process. Its primary objective is to generate an initial population of candidate individuals (chromosomes), which serves as a representation of the starting search space. We randomly generate an initial population based on the size of the genetic population, which is an input to the upper-layer algorithm. Random initialization allows for the exploration of a wide range of possible solutions, promoting diversity within the initial population. This diversity helps to prevent premature convergence to sub-optimal solutions and encourages the upper-layer algorithm to explore different regions of the search space.

Fitness function

The fitness function assesses the quality of individuals within the population, serving as a means to quantify the suitability of solutions for survival and reproduction during the evolutionary process in the upper-layer algorithm. Its principal objective is to assign a fitness value to each individual, reflecting its performance and guiding the search process. We define the fitness function as

$$f_u(\mathbf{X}, \mathbf{Y}) = \frac{1}{F(\mathbf{X}, \mathbf{Y})}, \quad (32)$$

which equates the minimization of the objective function to the maximization of fitness value. The rationale behind this transformation lies in the need to harmonize the upper-layer algorithm with the genetic algorithm framework, thereby fostering the evolutionary process and facilitating subsequent selection operations for the preferential reproduction of individuals with superior fitness.

Selection

The selection operation determines which individuals from the current population will be chosen as parents for the creation of offspring in the next generation. Its primary objective is to favor individuals with higher fitness values, thereby promoting the propagation of favorable traits and driving the population towards improved solutions. By favoring individuals with higher fitness, the upper-layer algorithm enhances the probability of identifying the optimal offloading location. The roulette wheel selection algorithm, employed in this paper, is based on the principle of proportional selection. It simulates a roulette wheel where each individual's fitness value corresponds to a portion of the wheel's circumference. The algorithm assigns a probability of selection to each individual that is proportional to its fitness value relative to the total fitness of the population. The probability of the i th individual $\mathbf{Y}[i]$ being selected is calculated by

$$P_i = \frac{f_u(\mathbf{X}, \mathbf{Y}[i])}{\sum_{j=1}^{NP_1} f_u(\mathbf{X}, \mathbf{Y}[j])}, \quad (33)$$

in which $f_u(\mathbf{X}, \mathbf{Y}[i])$ ($f_u(\mathbf{X}, \mathbf{Y}[j])$) is the fitness of the i th (j th) individual. A random number $\pi \in (0, 1)$ is generated to select the individual. The first individual $\mathbf{Y}[i]$ that satisfies condition $\sum_{j=1}^i P_j \geq \pi$, is selected.

Crossover

Selected parent individuals contribute their genetic information to create offspring individuals by crossover. The crossover process entails the exchange of genetic segments between parent individuals, resulting in the generation of novel combinations within the offspring individuals. The crossover facilitates the exploration of the solution space by generating offspring individuals that inherit advantageous traits from both parent individuals. The crossover operation initially identifies a gene segment comprised of one or more contiguous genes from each parent individual. Subsequently, this gene segment is inserted into the corresponding position within the opposite parent individual.

To determine whether a crossover operation should be performed, a random number within the range of 0 to 1 is generated and compared against the crossover probability P_c . A higher P_c contributes to increased diversity and exploration, aiding the upper-layer algorithm in discovering a greater number of possible solutions. However, a higher P_c may result in a slower convergence speed. A lower P_c can make the upper-layer algorithm more susceptible to getting trapped in local optima, consequently limiting its exploration capability. P_c serves as a critical parameter that substantially influences the performance of the upper-layer algorithm. Regrettably, the conventional P_c is fixed and lacks the ability to adapt to the iterations of the upper-layer algorithm.

Consequently, this paper incorporates an adaptive P_c expressed as

$$P_c = \begin{cases} P_{c1} - \frac{(P_{c1} - P_{c2})(f_u^h - f_u^{avg})}{f_u^{max} - f_u^{avg}}, & f_u^h \geq f_u^{avg} \\ P_{c1}, & f_u^h < f_u^{avg} \end{cases}, \quad (34)$$

in which P_{c1} and P_{c2} ($P_{c1} > P_{c2}$) are two constants, f_u^h denotes the higher fitness value among the two parent individuals, f_u^{avg} represents the average fitness value of the population, and f_u^{max} signifies the maximum fitness value within the population. In contrast to the conventional constant P_c , the adaptive P_c takes the shape of a right-angled trapezoid. This adaptive form reduces the probability of crossover when the fitness of the parent individuals is high, thereby effectively preserving individuals with superior fitness and promoting the retention of advantageous traits within the population.

Mutation

Mutation serves as a mechanism to maintain genetic variability within the population, preventing premature convergence to sub-optimal solutions. Through the mutation operation, a small random perturbation is applied to the genetic information of individuals, leading to the creation of new genetic variants. The mutation operation involves the selection of genes with a probability denoted as P_m , and a random number is generated within the range of 0 to 1 to determine the occurrence of gene mutation. Similarly, inappropriate P_m will also suppress the performance of upper-layer algorithm. A larger P_m facilitates enhanced exploration of the search space, introducing novel gene variants to the upper-layer algorithm. However, it may result in a slower convergence rate. Conversely, a smaller P_m will weaken the exploration capability of the upper-layer algorithm, leading to a propensity for local optima. Adaptive P_m is calculated by

$$P_m = \begin{cases} P_{m1} - \frac{(P_{m1}-P_{m2})(f_l-f_l^{avg})}{f_l^{max}-f_l^{avg}}, & f_l \geq f_l^{avg} \\ P_{m1}, & f_l < f_l^{avg} \end{cases}, \quad (35)$$

in which P_{m1} and P_{m2} ($P_{m1} > P_{m2}$) are two constant values, while f_l denotes the fitness of the parent individual. The remaining parameters remain consistent with those outlined in Equation (34).

Lower-layer algorithm based on the simulated annealing particle swarm optimization

Input: Size of the particle swarm population NP_2 ; Number of lower-layer iterations $Iter_2$; Task offloading location $\mathbf{Y}$; N tasks; M edge servers; Weight factors $\delta_1, \delta_2, \delta_3$.

Output: Optimal task offloading ratio $\mathbf{X}^b$.

```

1: Generate the initial population  $\mathbf{X}(0)[NP_2]$  and velocity  $\mathbf{V}(0)$  randomly;
2: Evaluate each individual from  $\mathbf{X}(0)[NP_2]$  by Equation (36) with  $\mathbf{Y}$ ;
3: Find the best  $\mathbf{X}^b$  from  $\mathbf{X}(0)[NP_2]$ ;
4: Copy  $\mathbf{X}(0)[NP_2]$  to  $\mathbf{X}^{pb}[NP_2]$ ;
5: for  $iter = 1$  to  $Iter_2$  do
6:   for  $j = 1$  to  $NP_2$  do
7:     Calculate  $\mathbf{V}^{new}$  by Equation (37);
8:     Calculate  $\mathbf{X}^{new}$  by Equation (38);
9:     Evaluate  $\mathbf{X}^{new}$  by Equation (36) with  $\mathbf{Y}$ ;
10:    if  $f_l(\mathbf{X}^{new}, \mathbf{Y}) < f_l(\mathbf{X}(iter-1)[j], \mathbf{Y})$  then
11:       $\mathbf{X}^{pb}[j] \leftarrow \mathbf{X}^{new}$ 
12:    end if
13:    Calculate  $P_{sa}(f_l(\mathbf{X}^{new}, \mathbf{Y}), f_l(\mathbf{X}(iter-1)[j], \mathbf{Y}))$  by Equation (40);
14:    if  $random(0, 1) < P_{sa}(f_l(\mathbf{X}^{new}, \mathbf{Y}), f_l(\mathbf{X}(iter-1)[j], \mathbf{Y}))$  then
15:       $\mathbf{V}(iter)[j] \leftarrow \mathbf{V}^{new}$ 
16:       $\mathbf{X}(iter)[j] \leftarrow \mathbf{X}^{new}$ 
17:    end if
18:  end for
19:  Find the best individual  $\mathbf{X}^b(iter)$  from  $\mathbf{X}(iter)[NP_2]$ ;
20:  if  $f_l(\mathbf{X}^b(iter), \mathbf{Y}) < f_l(\mathbf{X}^b, \mathbf{Y})$  then
21:     $\mathbf{X}^b \leftarrow \mathbf{X}^b(iter)$ ;
22:  end if
23: end for
24: return  $\mathbf{X}^b$ 

```

Algorithm 2. The lower-layer algorithm

Initialization and fitness function

The initialization of particle swarm optimization provides a suitable starting point for the subsequent iterative search of the lower-layer algorithm. The particle swarm optimization utilizes the movement of particles within a defined range to search for the optimum solution, making it particularly suitable for continuous optimization problems involving the offloading ratio at the lower layer. The initialization process involves the initialization of particle positions and particle velocities. This paper employs the random initialization method to generate the initial particle population and initial velocities. The particle position is constructed as N -dimensional to represent the offloading ratios of N tasks. The velocity of each particle is also N -dimensional, representing the rate of movement in each dimension for the particle.

The fitness function evaluates the quality of solutions in lower-layer algorithm by mapping each particle to a numerical value. It serves the purpose of assessing the quality of solutions. The fitness function of the lower-layer algorithm is defined as

$$f_l(\mathbf{X}, \mathbf{Y}) = F(\mathbf{X}, \mathbf{Y}). \quad (36)$$

A smaller fitness function value of a particle indicates a higher quality of its represented solution.

Update of individual and global best positions

The evaluation results of the fitness function are utilized to determine the individual best position for each particle and the global best position for the population. The individual best position corresponds to the solution associated with the historical extremum of the individual fitness, while the global best position corresponds to the solution associated with the historical extremum of the population's fitness. By updating the individual and global best positions, the lower-layer algorithm is able to progressively optimize solutions within the search

space, enabling individuals and the entire particle swarm to approach their optimums gradually. This updating mechanism guides particles to search in more favorable solution spaces, thereby enhancing the convergence and search efficiency of the lower-layer algorithm.

When updating the individual best position, a comparison is made between the current solution's fitness and the historical individual best fitness value to determine whether an update is necessary. Let $X^{pb}[j]$ represent the individual best position of the j th particle, and $X^{(iter)}[j]$ denote the position of the j th particle in the $iter$ th iteration. If $f_i(X^{(iter)}[j], Y) < f_i(X^{pb}[j], Y)$, it signifies that the current position of the particle is better than its historical best position, prompting an update from $X^{pb}[j]$ to $X^{(iter)}[j]$. The purpose of updating the individual best position is to ensure that each particle maintains tracking of its individual optimal solution and performs more targeted searches in subsequent iterations.

When updating the global best position, the comparison between the best fitness value of the current population and the historical global best fitness value is performed to determine whether an update to the global best position is required. Let X^b denote the global best position, and $X^{b(iter)}$ represent the best position of the population in the $iter$ th iteration. If $f_i(X^{b(iter)}, Y) < f_i(X^b, Y)$, it indicates that the best position of the current population is better than the historical global best position, thereby triggering an update from X^b to $X^{b(iter)}$. The purpose of updating the global best position is to maintain tracking of the optimal solution for the entire particle swarm and provide guidance throughout the entire search process.

Update of particle velocity and position

The purpose of velocity updating is to guide particles in exploring the search space and encourage them to approach their individual best position and the global best position, thereby assisting the lower-layer algorithm in obtaining superior solutions. When updating the velocity, the current velocity, individual best position, and global best position are taken into account. Subsequently, the particle's velocity is updated by applying appropriate factors such as the inertia factor, learning factor, and random factor. This integration of factors facilitates a balanced adjustment that considers the particle's inherent motion tendency, the guidance provided by both the individual and global best positions, and the introduction of controlled randomness to enhance the overall search capability. Through the comprehensive utilization of these factors, particles are able to conduct targeted exploration within the search space, with the ultimate goal of discovering superior solutions. Velocity of the j th particle in the $iter$ th iteration is updated by

$$V^{(iter)}[j] = \chi^{(iter)} V^{(iter-1)}[j] + c_1 \vartheta_1 (X^{pb(iter-1)}[j] - X^{(iter-1)}[j]) + c_2 \vartheta_2 (X^{b(iter-1)}[j] - X^{(iter-1)}[j]), \quad (37)$$

in which $\chi^{(iter)}$ denotes the inertia factor, ϑ_1 and ϑ_2 are random numbers between 0 and 1, c_1 and c_2 are learning factors. The inertia factor serves as an indicator of particles' inherent motion tendency, while the learning factor is utilized to facilitate behavioral adjustments based on the individual and global best positions. Furthermore, the random factor introduces a controlled level of stochasticity to enhance the global search capability of the lower-layer algorithm. Position of the j th particle in the $iter$ th iteration is updated by

$$X^{(iter)}[j] = X^{(iter-1)}[j] + V^{(iter)}[j]. \quad (38)$$

The inertia factor reflects the influence of the previous iteration's velocity on the current particle velocity. A higher value of the inertia factor strengthens the global optimization capability of the lower-layer algorithm while weakening the local optimization capability. Conversely, a lower value of the inertia factor weakens the global optimization capability and enhances the local optimization capability of the lower-layer algorithm. This paper employs a dynamic inertia factor, expressed as

$$\chi^{(iter)} = \chi_1 - \frac{(\chi_1 - \chi_2) iter}{Iter}, \quad (39)$$

in which χ_1 represents the maximum inertia factor, χ_2 represents the minimum inertia factor, $iter$ represents the current number of iterations, and $Iter$ represents the maximum number of iterations. The dynamic inertia factor offers a greater flexibility for the lower-layer algorithm. As the number of iterations increases, the dynamic inertia factor progressively decreases. Consequently, the lower-layer algorithm exhibits a strong global search capability in the early stages, enabling exploration of previously unexplored regions. In the later stages, it demonstrates a strong local optimization capability and enables a fine-grained search.

A decision criterion based on simulated annealing is utilized to determine the necessity of performing updates. Simulated annealing introduces a mechanism for escaping local optima by allowing occasional acceptance of worse particles. This promotes exploration of the search space and prevents premature convergence to sub-optimal solutions.

$$P_{sa}(f'_l, f_l) = \begin{cases} 1, & f'_l \leq f_l \\ e^{-\frac{f'_l - f_l}{T_{iter}}}, & f'_l > f_l \end{cases}, \quad (40)$$

in which f'_l denotes the fitness of the new particle position, f_l denotes the fitness of the current particle position, T_{iter} denotes the current temperature. A random number $\pi \in (0, 1)$ is generated and compared with $P_{sa}(f'_l, f_l)$. If $\pi < P_{sa}(f'_l, f_l)$, the new particle position is accepted and updated to become the latest position of the particle.

Experiment and analysis

The experiments were conducted on a PC equipped with the Intel Core i5 CPU (2 cores, 2.3 GHz), 16 GB of RAM, and a 64-bit operating system. The algorithms were implemented in Java using the JDK 1.8 development environment. To ensure the robustness of our evaluations, the experimental parameters were generated randomly within specified ranges rather than using fixed values. This approach captures the variability in system parameters and allows for testing under a wide range of scenarios, providing a comprehensive assessment of the established offloading model and the proposed offloading algorithm. In this study, we utilized synthetic datasets specifically designed to simulate realistic industrial Internet tasks, encompassing a wide range of resource demands and execution constraints. The synthetic data were carefully crafted based on extensive practical experience to ensure they accurately reflect real-world conditions. By randomly generating the experimental parameters within specified ranges, we covered various potential operational conditions, thereby enhancing the reliability of our evaluations.

The evaluation of the system model section first analyzes the impact of varying system parameters on the objective values, as solved by the proposed offloading algorithm. This provides insight into how different system configurations affect the performance of the established model. Following this, in the evaluation of the offloading algorithm section, we conduct a performance comparison between our offloading algorithm and other offloading algorithms to assess its effectiveness. The default experimental parameter settings used for these evaluations are summarized in Table 4.

Evaluation of the system model

We first evaluate the impact of different offloading modes on the task execution performance. According to the system architecture, there are three offloading modes:

- (1) Edge: All tasks are offloaded to edge servers for execution.
- (2) Local: All tasks are executed on the industrial terminals.
- (3) Local-Edge: The execution of tasks can be performed either locally on industrial terminals or offloaded to edge servers. The offloading strategy is provided by the approach presented in this paper.

Figure 2 shows the objective values under different offloading modes. It can be observed that the objective value of Local-Edge mode is the minimum, while the objective value of Local mode is the maximum, with the objective value of Edge mode falling between the two extremes. This phenomenon signifies that offloading tasks to external platforms has the possibility to improve task execution efficiency. However, the achievement of such gains necessitates the implementation of precise offloading strategies. Incorrect offloading strategies can lead to misguidance in the task offloading process, consequently yielding unfavorable outcomes. In comparison to Edge and Local modes, the objective value of Local-Edge mode exhibits reductions of 27% and 46%, respectively. It is evident that blindly offloading all tasks to edge servers is not the optimal strategy. The underlying reason is that

Parameter	Value
N	[60,140]
M	[3,9]
f_i^l	3GHz
f_j^e	10GHz
p_{ij}	100 mW
ζ	−100 dBm
Ψ_j	2048 MIPS
Ω_j	1024 MB
Λ	20 MHz
$\alpha_1 \sim \alpha_3$	1/3
η_k	1/3
$\delta_1 \sim \delta_3$	1/3
θ	[0.2,0.8]
P_{c1}	0.9
P_{c2}	0.6
P_{m1}	0.1
P_{m2}	0.001

Table 4. Experimental parameter settings.

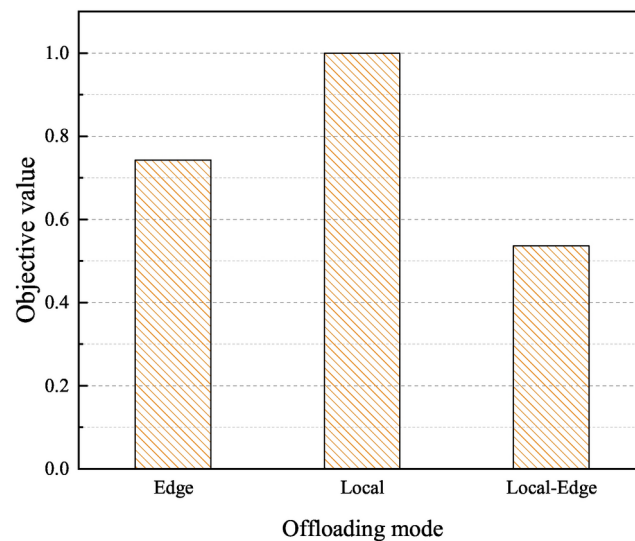


Fig. 2. Objective values under different offloading modes.

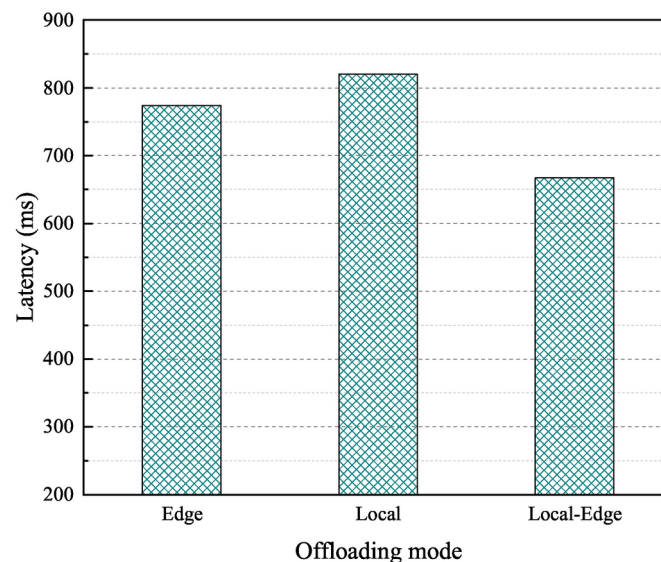


Fig. 3. Latencies under different offloading modes.

offloading tasks to edge servers introduces additional consumptions attributed to task data transmission. In the absence of precise offloading decisions, the potential decrease in objective value achieved by task offloading may be offset by the overhead incurred from data transmission.

The objective function comprises three sub-objectives: latency, security risk, and load variance. Figure 3 demonstrates the optimization effect of the sub-objective of latency, serving as an example to illustrate the performance improvement of the sub-objectives. The results reveal that the latency of the Local mode exhibits the highest value, reaching 820ms. The Edge mode demonstrates a slight reduction in latency, with a measured value of 774ms. The Local-Edge mode yields the lowest latency, recorded at 667ms. Compared to the Local mode and Edge mode, the Local-Edge mode achieves reductions in latency of 18.7% and 13.8%, respectively. The results indicate that task offloading indeed contributes to the reduction of task execution latency. Our model will offload tasks to appropriate locations for execution based on the optimization objective. However, it is crucial to emphasize that the accuracy of offloading strategy significantly impacts the offloading performance.

Figure 4 shows objective values under different numbers of edge servers. The observed trend reveals a gradual decrease in the objective values as the number of edge servers increases. The reduction in objective values from 3 edge servers to 9 edge servers is 13.22%. This indicates that scaling up the edge server quantity positively impacts the offloading performance. The decreasing trend in objective values can be attributed to the distribution of tasks among a larger number of edge servers, resulting in a more balanced load and efficient execution of tasks. The availability of additional edge servers enhances the system's capability to offload tasks efficiently, thereby reducing

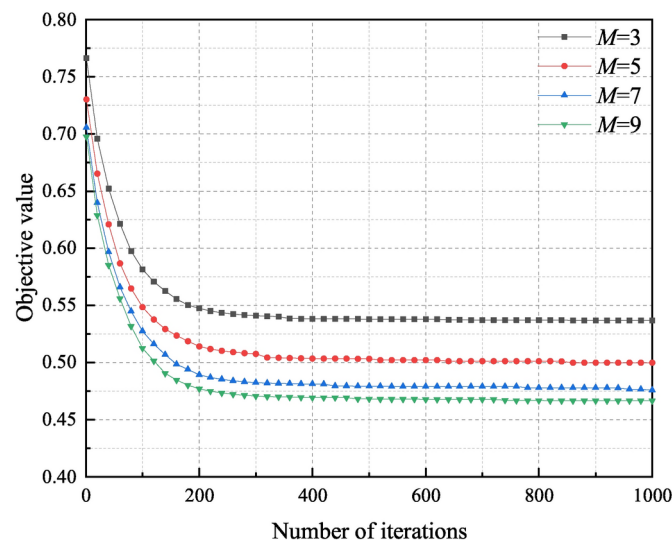


Fig. 4. Objective values under different numbers of edge servers.

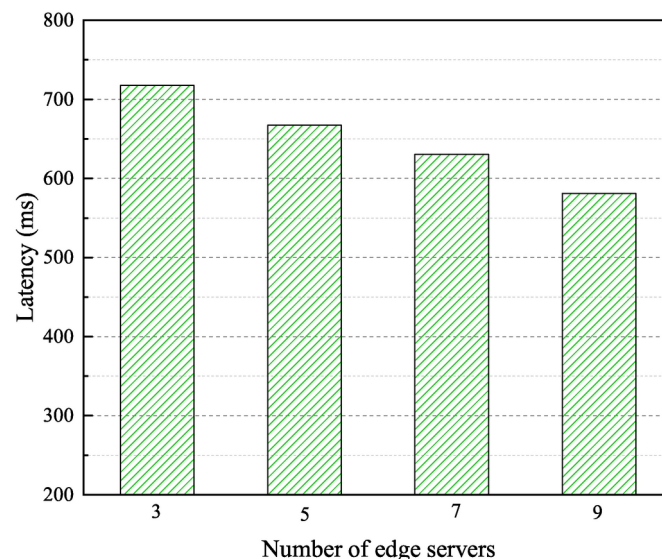


Fig. 5. Latencies under different numbers of edge servers.

both latency and load variance. However, the decrease in objective values demonstrates diminishing returns as the number of edge servers increases. For example, the reduction in objective values from 3 edge servers to 5 edge servers is 6.9%, while the decrease from 5 edge servers to 7 edge servers is 4.84%, and finally, the decrease from 7 edge servers to 9 edge servers is only 1.95%. This phenomenon indicates that increasing the number of edge servers does not lead to an infinite improvement in task offloading performance. The underlying reason is that when a sufficient number of edge servers are available, the limiting factor for offloading performance shifts away from the availability of edge resources.

Figure 5 shows latencies under different numbers of edge servers. It can be observed that there is a decreasing trend in the task execution latencies as the number of edge servers increases. This suggests that increasing the number of edge servers has a positive impact on the system's performance in terms of reducing task execution latency. Specifically, as the number of edge servers increases from 3 to 9, there is a 19.02% decrease in latency. The substantial reduction in task execution latency indicates that utilizing more edge servers leads to significant improvements in task processing efficiency. As the number of edge servers increases, the available resources for task execution also increase. The proposed offloading approach can avoid the pressure of resource capacity constraints, allowing for optimized task offloading with minimized latency. This finding provides experimental evidence and reference for enhancing the efficiency of EC systems and ensuring timely task execution.

Figure 6 illustrates the comparison of objective values with and without load balancing. The results demonstrate that the final objective value is 0.628 without load balancing, whereas with load balancing, the final

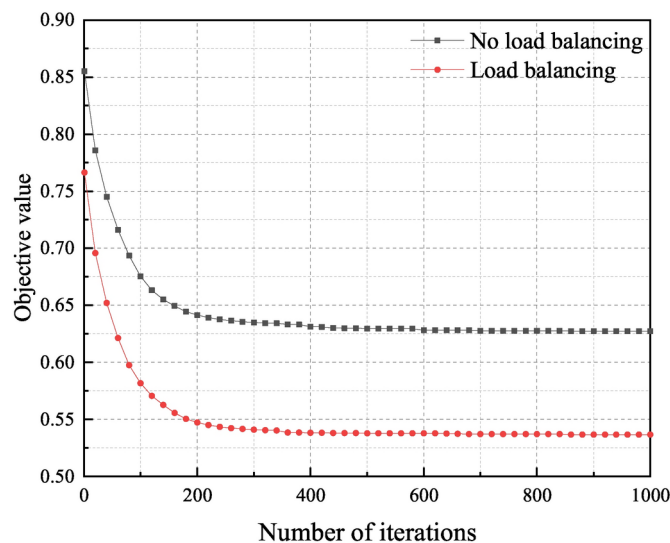


Fig. 6. Objective values with and without load balancing.

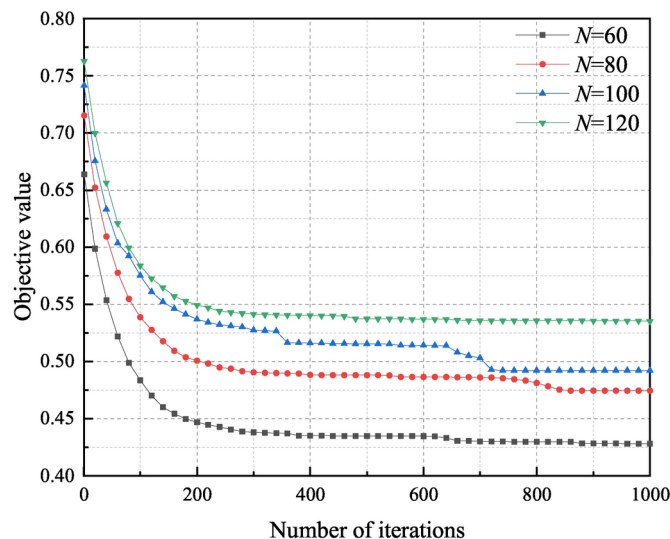


Fig. 7. Objective values under different numbers of industrial terminals.

objective value is reduced to 0.537. In the absence of load balancing, offloading tasks among edge servers may be uneven, leading to resource overutilization or underutilization. The presence of load imbalance will lead to the non-offloading of some tasks, thereby increasing the objective value. This discrepancy can lead to a decline in EC performance and may potentially diminish the effectiveness of task execution. Load balancing plays a crucial role in offloading the tasks evenly across multiple edge servers. The experimental results indicate that the load balancing in our model effectively mitigates this issue.

Figure 7 shows objective values under different numbers of industrial terminals. The results demonstrate an increasing trend in the objective value with an increase in the number of industrial terminals. Specifically, when the number of industrial terminals increased from 60 to 120, there is a 25% growth in the objective value. The fundamental cause of this phenomenon is attributed to the escalating number of industrial terminals, which consequently escalates both the task and resource demand. The availability of resources is relatively higher when the number of industrial terminals is small, enabling the system to better meet the offloading requirements of tasks, resulting in lower objective values. However, as the number of industrial terminals increases, the availability of resources decreases, intensifying the competition among tasks and leading to an increase in the objective value. The constrained task offloading and the subsequent rise in objective values stem from the system's limited resources, which hinder the simultaneous fulfillment of all task requirements.

Figure 8 shows load variances under different numbers of industrial terminals. It can be observed that with the increase in the number of industrial terminals, the load variance on edge servers gradually increases. As the number of industrial terminals increases from 60 to 140, there is an upward trend in the load variance, rising

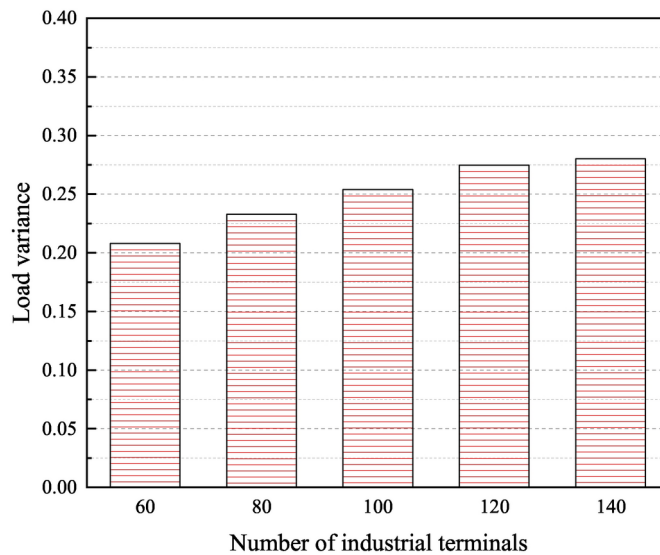


Fig. 8. Load variances under different numbers of industrial terminals.

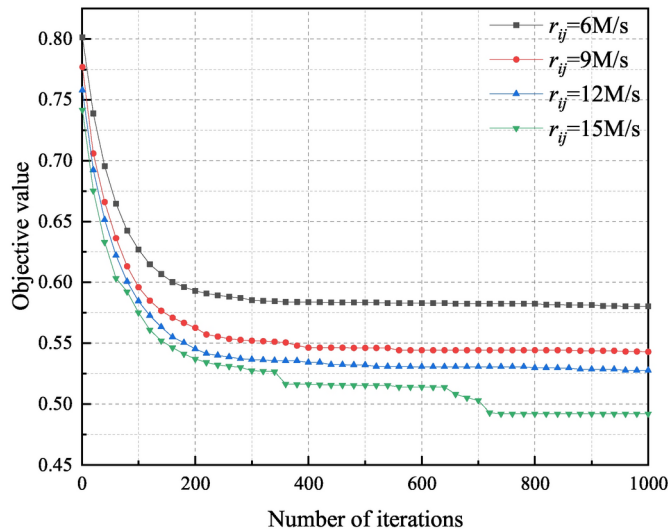


Fig. 9. Objective values under different wireless network rates.

from 0.208 to 0.280. With the increase in the number of industrial terminals, the competition for edge server resources becomes more pronounced. Under the constraint of a fixed number of edge servers, the availability of edge resources is limited, and the system needs to prioritize the execution of a greater number of tasks. As the number of industrial terminals continues to rise, edge servers experience an increased load pressure. This results in a wider distribution of the edge server load and a gradual increase in load variance. The experiment demonstrates that, alongside load balancing, as the number of industrial terminals continues to increase, the addition of edge servers should also be considered to uphold EC efficiency.

Figure 9 shows objective values under different wireless network rates. It can be observed that as the wireless network rate increases, the objective value continuously decreases. When the wireless network rate increases from 6M/s to 15M/s, the objective value decreases by 15.19%. This suggests that higher wireless network rates contribute to the improved performance and efficiency in achieving the desired objective. In order to achieve an improvement in task offloading performance, it is necessary for the data transmission consumption during offloading process to be lower than the execution consumption after the tasks are offloaded. As the wireless network rate increases, the data transmission consumption consistently decreases, thereby amplifying the reduction in objective value resulting from task offloading. This experiment highlights the significance of wireless networks in task offloading and further demonstrates the positive impact of fast network rates on the objective value.

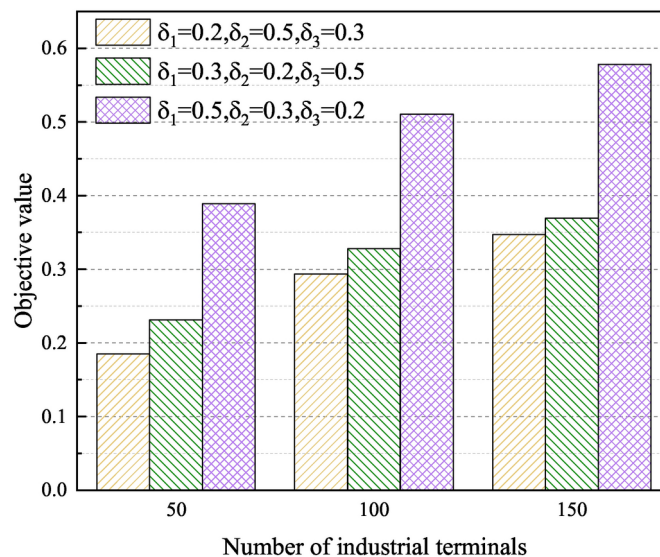


Fig. 10. Objective values under different weights.

By adjusting the weights of three sub-objectives, the effect of different weights on the objective value can be observed. The results are shown in Fig. 10. It can be seen that the objective value exhibits the smallest value for the first set of weights ($\delta_1 = 0.2, \delta_2 = 0.5, \delta_3 = 0.3$), an intermediate value for the second set of weights ($\delta_1 = 0.3, \delta_2 = 0.2, \delta_3 = 0.5$), and the largest value for the third set of weights ($\delta_1 = 0.5, \delta_2 = 0.3, \delta_3 = 0.2$). The first set of weights is characterized by a higher proportion of security risks, the second set of weights exhibits a higher proportion of load variance, and the third set of weights is associated with a higher proportion of latency. This observation indicates that the value of latency sub-objective is relatively large. As the weight assigned to sub-objective latency becomes larger, it contributes more heavily to the calculation of the weighted objective value. The experimental findings suggest that our model enables users to freely adjust weights, allowing for customized task offloading based on their individual needs.

Evaluation of the offloading algorithm

In this section, we present a comparative evaluation for the proposed offloading algorithm against existing offloading algorithms to validate its performance. The conducted experiments in this section encompass the assessment of six distinct offloading algorithms: (1) our proposed bi-layer offloading algorithm with joint load balance and fuzzy security (BOALBFS), which is based on the adaptive genetic algorithm and simulated annealing particle swarm optimization; (2) the offloading algorithm based on the genetic algorithm and particle swarm optimization (GAPSO); (3) the offloading algorithm based on the simulated annealing and particle swarm optimization (SAPSO); (4) the offloading algorithm based on the hill climbing and particle swarm optimization (HCPSO); (5) the offloading algorithm based on the grey wolf optimization and particle swarm optimization (GWOPSO); (6) the offloading algorithm based on the improved whale optimization and particle swarm optimization (IWOPSO).

Figure 11 shows the performance comparison of offloading algorithms under different numbers of iterations. The results demonstrate that with an increasing number of iterations, offloading algorithms converge to their stable objective values. Evidently, the final objective value of BOALBFS is the smallest, indicating its superior performance compared to other offloading algorithms. In contrast, HCPSO demonstrates the highest final objective value, indicating its comparatively poorer performance among the evaluated offloading algorithms. In terms of quantitative analysis, BOALBFS achieves a reduction in the objective value compared to algorithms HCPSO, SAPSO, GWOPSO, GAPSO, and IWOPSO, with respective decreases of 28.04%, 22.24%, 14.16%, 10.34%, and 7.80%. The utilization of a local optimization heuristic algorithm in the upper layer of HCPSO prevents it from performing global search, resulting in being trapped in local optima and ultimately leading to a higher objective value. The observations reveal that IWOPSO and GAPSO exhibit premature convergence, while SAPSO and GWOPSO show relatively slower convergence but lower search efficiency. Although IWOPSO, GAPSO, SAPSO, and GWOPSO possess the capabilities for global optimization, their achieved objective values surpass that of HCPSO but are inferior to BOALBFS. This disparity arises due to the augmented global optimization ability of AGAPSO, which benefits from the integration of adaptive mechanism, simulated annealing, genetic algorithm, and particle swarm optimization.

Figure 12 presents a comparison of the running time among different offloading algorithms. It is evident that HCPSO demonstrates the shortest running time, with values that are a mere fraction of BOALBFS, GAPSO, SAPSO, GWOPSO, and IWOPSO. Specifically, HCPSO's running time is only 27.63%, 25.54%, 22.57%, 24.20%, and 26.44% of the running times of BOALBFS, GAPSO, SAPSO, GWOPSO, and IWOPSO. This phenomenon can be attributed to the absence of iterative search in the upper-layer algorithm of HCPSO. The non-iterative nature of HCPSO's upper-layer algorithm allows it to make faster decisions, resulting in a significant reduction

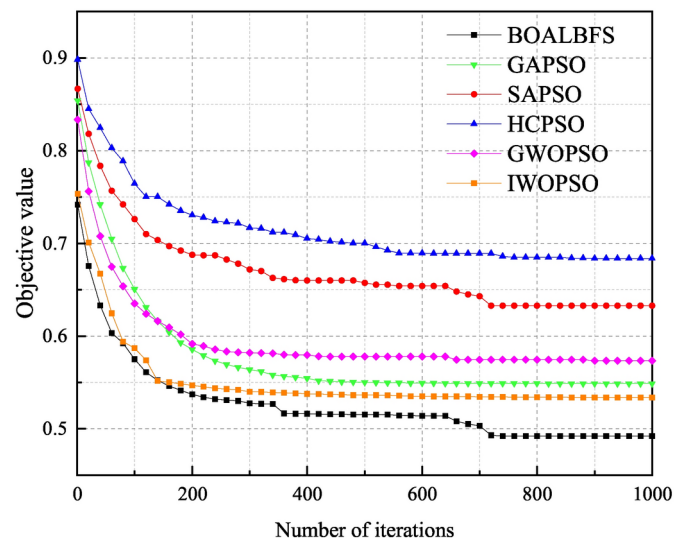


Fig. 11. Performance comparison of offloading algorithms under different numbers of iterations.

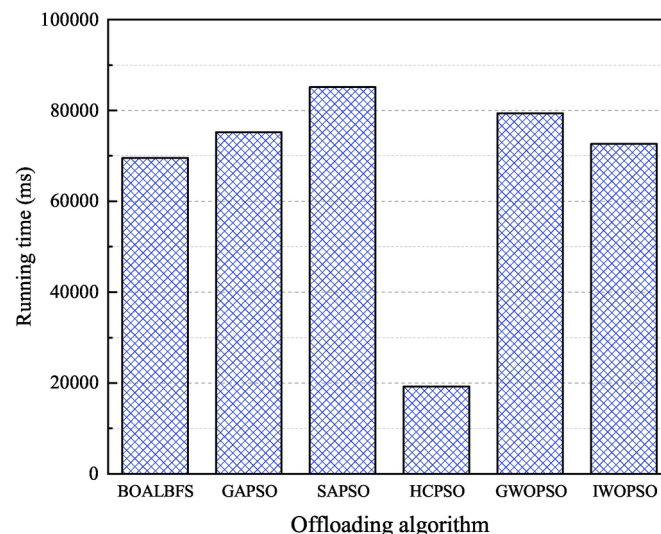


Fig. 12. Comparison of the running time of offloading algorithms.

in the overall computational effort required to end the running process. However, as depicted in Fig. 11, HCPSO exhibits the lowest accuracy. The running time of BOALBFS is found to be higher than that of HCPSO but lower than the running times of GAPSO, SAPSO, GWOPSO, and IWOPSO. This observation highlights the advantage of BOALBFS, which achieves the highest level of accuracy with a comparatively lower running time. While BOALBFS achieves the highest accuracy, there are scenarios where faster running speed takes precedence over precision. In such cases, BOALBFS offers flexibility by reducing the number of iterations, enabling quicker results with minimal impact on accuracy. This adaptability makes it suitable for real-world applications where timely decisions are crucial, such as in time-sensitive industrial processes. Additionally, algorithms like HCPSO, with naturally shorter running times, can be employed when speed is the priority. Though HCPSO sacrifices some accuracy, its fast execution makes it ideal for tasks where approximate solutions are acceptable, such as in emergencies or high-priority operations.

To thoroughly validate the proposed offloading algorithm, a comprehensive comparison is conducted with existing algorithms across various scenarios. In these scenarios, the wireless network rate changes from 4M/s to 10M/s. Table 5 presents the latencies of offloading algorithms across different scenarios, while their performance is visually depicted using box plots in Fig. 13. The results reveal that BOALBFS consistently achieves the minimum latency compared to other algorithms, indicating its superior performance across various scenarios. This consistent minimization of latency underscores the robustness and efficiency of BOALBFS in handling diverse operational conditions. In relation to BOALBFS, GAPSO, SAPSO, HCPSO, GWOPSO, and IWOPSO exhibit relative errors of 5.90%, 9.32%, 13.25%, 3.94%, and 2.82% respectively. The presence of positive error

Rate						
Algorithm	BOALBFS	GAPSO	SAPSO	HCPSO	GWOPSO	IWOPSO
4M/s	638.33	651.58	685.21	706.32	645.24	648.29
6M/s	590.72	630.64	650.45	685.2	618.23	602.36
8M/s	561.12	600.14	612.57	635.84	594.76	589.13
10M/s	537.31	582.47	596.13	608.46	561.04	553.42

Table 5. Latencies (ms) of offloading algorithms in different scenarios.

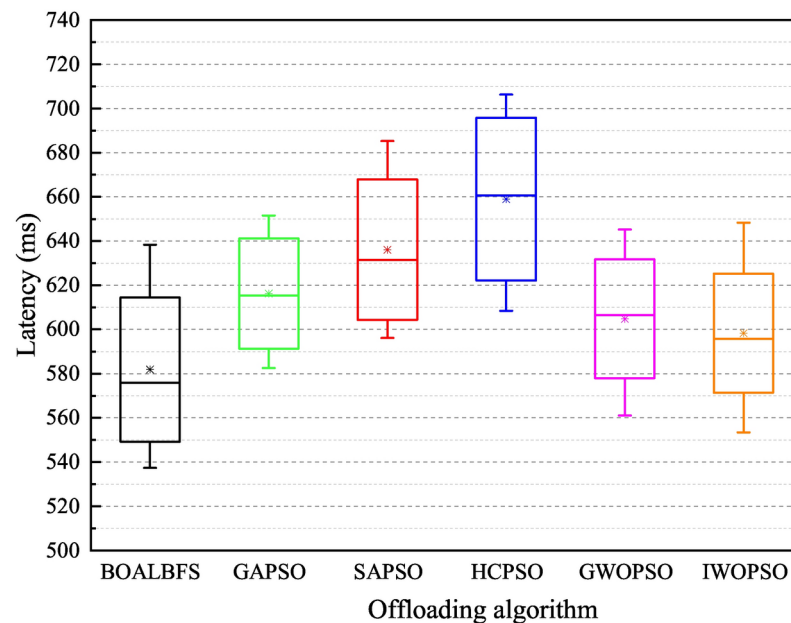


Fig. 13. Latencies of offloading algorithms in different scenarios.

values signifies BOALBFS's higher degree of accuracy, thus affirming its efficacy in optimizing task offloading. The substantial reduction in latency is attained exclusively when the offloading algorithm demonstrates a sufficiently high level of accuracy. In contrast, inaccurate offloading algorithms compromise the system performance and fail to achieve significant improvements in latency reduction. Consistent with the outcomes depicted in Fig. 9, the observations reveal a discernible trend where the reduction in latency is consistently observed with the progressive improvement of wireless network rates. The underlying reason for this phenomenon remains consistent, as the continuous increase in wireless network rate effectively reduces data transmission latency, consequently leading to a reduction in the overall latency of task offloading.

Conclusion

This paper aims to address the problem of latency-aware multi-server partial task offloading for EC in the industrial Internet by integrating joint load balancing and security protection. A novel task offloading model and a bi-layer offloading algorithm were presented. We established a constrained optimization model to mathematically formalize the task offloading problem for EC in the industrial Internet, providing a quantitative foundation for investigating it. The NP-hardness of the established model was rigorously demonstrated through a formal proof. To solve the composite offloading strategy, comprising both the offloading location and offloading ratio derived from the NP-hard model, we proposed a bi-layer offloading algorithm with joint load balance and fuzzy security, which is based on the adaptive genetic algorithm and simulated annealing particle swarm optimization. The upper-layer algorithm was responsible for determining the appropriate edge server to which the task should be offloaded, while the lower-layer algorithm determined the optimal ratio of task offloading. Drawing upon extensive experimental results, it was found that the evaluation outcomes of the established model aligned with the expected results. The proposed offloading algorithm was shown to outperform existing algorithms in terms of solution accuracy, demonstrating its superior performance. In the future, collaboration among multiple industrial terminals and competition among multiple edge servers will be considered based on this work. The expanded problem will allow tasks to be offloaded among industrial terminals and will support games among edge servers. To address this problem, a game theory-based model incorporating "terminal-to-terminal offloading" and "terminal-to-edge offloading" will be established, while

accounting for interest competition among edge servers. This model will be more complex and challenging to solve, and parallel offloading algorithms will be explored to efficiently address its computational complexity.

Data availability

The datasets used and/or analyzed during the current study available from the corresponding author on reasonable request.

Received: 7 September 2024; Accepted: 8 November 2024

Published online: 13 November 2024

References

- Sharma, M., Tomar, A. & Hazra, A. Edge computing for industry 5.0: Fundamental, applications and research challenges. *IEEE Int. Things J.* **11**, 19070–19093. <https://doi.org/10.1109/JIOT.2024.3359297> (2024).
- Nain, G., Pattanaik, K. & Sharma, G. Towards edge computing in intelligent manufacturing: Past, present and future. *J. Manuf. Syst.* **62**, 588–611. <https://doi.org/10.1016/j.jmsys.2022.01.010> (2022).
- Shakarami, A., Ghobaei-Arani, M. & Shahidinejad, A. A survey on the computation offloading approaches in mobile edge computing: A machine learning-based perspective. *Comput. Netw.* **182**, 107496. <https://doi.org/10.1016/j.comnet.2020.107496> (2020).
- Akbar, H., Zubair, M. & Malik, M. S. The security issues and challenges in cloud computing. *Int. J. Electron. Crime Invest.* **7**, 13–32. <https://doi.org/10.54692/ijeci.2023.0701125> (2023).
- Das, R. & Inuwa, M. M. A review on fog computing: Issues, characteristics, challenges, and potential applications. *Telemat. Inform. Rep.* **10**, 100049. <https://doi.org/10.1016/j.teler.2023.100049> (2023).
- Maleki, E. F., Ma, W., Mashayekhy, L. & Roche, H. L. QoS-aware content delivery in 5G-enabled edge computing: Learning-based approaches. *IEEE Trans. Mobile Comput. (Early Access)*[SPACE]<https://doi.org/10.1109/TMC.2024.3363143> (2024).
- Baranwal, G., Kumar, D. & Vidyarthi, D. P. Blockchain based resource allocation in cloud and distributed edge computing: A survey. *Comput. Commun.* **209**, 469–498. <https://doi.org/10.1016/j.comcom.2023.07.023> (2023).
- Gong, Y. et al. Computation offloading and quantization schemes for federated Satellite-Ground Graph Networks. *IEEE Trans. Wireless Commun.* **23**, 14140–14154. <https://doi.org/10.1109/TWC.2024.3409691> (2024).
- Shakarami, A., Shahidinejad, A. & Ghobaei-Arani, M. An autonomous computation offloading strategy in mobile edge computing: A deep learning-based hybrid approach. *J. Netw. Comput. Appl.* **178**, 102974. <https://doi.org/10.1016/j.jnca.2021.102974> (2021).
- Sun, G., Xu, Z., Yu, H. & Chang, V. Dynamic network function provisioning to enable network in box for industrial applications. *IEEE Trans. Industr. Inf.* **17**, 7155–7164. <https://doi.org/10.1109/TII.2020.3042872> (2021).
- Sun, G., Zhang, Y., Yu, H., Du, X. & Guizani, M. Intersection fog-based distributed routing for V2V communication in urban vehicular Ad Hoc networks. *IEEE Trans. Intell. Transp. Syst.* **21**, 2409–2426. <https://doi.org/10.1109/TITS.2019.2918255> (2020).
- Zhang, J., Gong, B., Waqas, M., Tu, S. & Han, Z. A hybrid many-objective optimization algorithm for task offloading and resource allocation in multi-server mobile edge computing networks. *IEEE Trans. Serv. Comput.* **16**, 3101–3114. <https://doi.org/10.1109/TSC.2023.3268990> (2023).
- Ke, H., Wang, H., Sun, W. & Sun, H. Adaptive computation offloading policy for multi-access edge computing in heterogeneous wireless networks. *IEEE Trans. Netw. Serv. Manage.* **19**, 289–305. <https://doi.org/10.1109/TNSM.2021.3118696> (2022).
- Zhang, W. et al. Secure and optimized load balancing for multitier IoT and edge-cloud computing systems. *IEEE Internet Things J.* **8**, 8119–8132. <https://doi.org/10.1109/JIOT.2020.3042433> (2021).
- Fan, Q. & Ansari, N. Towards workload balancing in fog computing empowered IoT. *IEEE Trans. Network Sci. Eng.* **7**, 253–262. <https://doi.org/10.1109/TNSE.2018.2852762> (2020).
- Tange, K., De Donno, M., Fafoutis, X. & Dragoni, N. A systematic survey of industrial Internet of things security: Requirements and fog computing opportunities. *IEEE Commun. Surv. Tutor.* **22**, 2489–2520. <https://doi.org/10.1109/COMST.2020.3011208> (2020).
- Li, T., He, X., Jiang, S. & Liu, J. A survey of privacy-preserving offloading methods in mobile-edge computing. *J. Netw. Comput. Appl.* **203**, 103395. <https://doi.org/10.1016/j.jnca.2022.103395> (2022).
- Yi, B., Cao, Y. P. & Song, Y. Network security risk assessment model based on fuzzy theory. *J. Intell. Fuzzy Syst.* **38**, 3921–3928. <https://doi.org/10.3233/JIFS-179617> (2020).
- Mehta, P., Kumar, S., Tejani, G. G. & Khishe, M. MOBO: A multiobjective brown bear optimization algorithm for solving constrained structural optimization problems. *J. Optim.* **2024**, 5546940. <https://doi.org/10.1155/2024/5546940> (2024).
- Mashru, N., Tejani, G. G., Patel, P. & Khishe, M. Optimal truss design with MOHO: A multi-objective optimization perspective. *PLoS ONE* **19**, e0308474. <https://doi.org/10.1371/journal.pone.0308474> (2024).
- Kumar, S. et al. Optimization of truss structures using multi-objective cheetah optimizer. *Mech. Des. Struct. Mach. (Early Access)*[SPACE]<https://doi.org/10.1080/15397734.2024.2389109> (2024).
- You, Q. & Tang, B. Efficient task offloading using particle swarm optimization algorithm in edge computing for industrial internet of things. *J. Cloud Comput.* **10**, 1–11. <https://doi.org/10.1186/s13677-021-00256-4> (2021).
- Zhou, Z. et al. Learning-based URLLC-aware task offloading for Internet of health things. *IEEE J. Sel. Areas Commun.* **39**, 396–410. <https://doi.org/10.1109/JSAC.2020.3020680> (2021).
- Jiao, X. et al. Deep reinforcement learning for time-energy tradeoff online offloading in MEC-enabled industrial Internet of things. *IEEE Trans. Netw. Sci. Eng.* **10**, 3465–3479. <https://doi.org/10.1109/TNSE.2023.3263169> (2023).
- Deng, X. et al. Intelligent delay-aware partial computing task offloading for multiuser industrial Internet of things through edge computing. *IEEE Internet Things J.* **10**, 2954–2966. <https://doi.org/10.1109/JIOT.2021.3123406> (2023).
- Liu, T., Zhang, Y., Zhu, Y., Tong, W. & Yang, Y. Online computation offloading and resource scheduling in mobile-edge computing. *IEEE Internet Things J.* **8**, 6649–6664. <https://doi.org/10.1109/JIOT.2021.3051427> (2021).
- Dai, X. et al. Task co-offloading for D2D-assisted mobile edge computing in industrial Internet of things. *IEEE Trans. Industr. Inf.* **19**, 480–490. <https://doi.org/10.1109/TII.2022.3158974> (2023).
- Guo, M. et al. HAGP: a heuristic algorithm based on greedy policy for task offloading with reliability of MDs in MEC of the industrial Internet. *Sensors* **21**, 3513. <https://doi.org/10.3390/s21103513> (2021).
- Saleem, U., Liu, Y., Jangsher, S., Tao, X. & Li, Y. Latency minimization for D2D-enabled partial computation offloading in mobile edge computing. *IEEE Trans. Veh. Technol.* **69**, 4472–4486. <https://doi.org/10.1109/TVT.2020.2978027> (2020).
- Sun, G. et al. Profit maximization of independent task offloading in MEC-enabled 5G Internet of vehicles. *IEEE Trans. Intell. Transp. Syst. (Early Access)*[SPACE]<https://doi.org/10.1109/TITS.2024.3416300> (2024).
- Sun, G., Wang, Y., Yu, H. & Guizani, M. Proportional fairness-aware task scheduling in space-air-ground integrated networks. *IEEE Trans. Serv. Comput. (Early Access)*[SPACE]<https://doi.org/10.1109/TSC.2024.3478730> (2024).
- Sun, G., Sheng, L., Luo, L. & Yu, H. Game theoretic approach for multipriority data transmission in 5G vehicular networks. *IEEE Trans. Intell. Transp. Syst.* **23**, 24672–24685. <https://doi.org/10.1109/TITS.2022.3198046> (2022).

33. Mondal, S., Das, G. & Wong, E. A game-theoretic approach for non-cooperative load balancing among competing cloudlets. *IEEE Open J. Commun. Soc.* **1**, 226–241. <https://doi.org/10.1109/OJCOMS.2020.2971613> (2020).
34. Chen, W., Zhu, Y., Liu, J. & Chen, Y. Enhancing mobile edge computing with efficient load balancing using load estimation in ultra-dense network. *Sensors* **21**, 3135. <https://doi.org/10.3390/s21093135> (2021).
35. Xu, C. et al. Dynamic parallel multi-server selection and allocation in collaborative edge computing. *IEEE Trans. Mobile Comput. (Early Access)*[SPACE]<https://doi.org/10.1109/TMC.2024.3376550> (2024).
36. Lu, Y., Han, D., Wang, X. & Gao, Q. Enhancing vehicular edge computing system through cooperative computation offloading. *Clust. Comput.* **26**, 771–788. <https://doi.org/10.1007/s10586-022-03803-z> (2023).
37. Yan, L., Chen, H., Tu, Y. & Zhou, X. A task offloading algorithm with cloud edge jointly load balance optimization based on deep reinforcement learning for unmanned surface vehicles. *IEEE Access* **10**, 16566–16576. <https://doi.org/10.1109/ACCESS.2022.3150406> (2022).
38. Lu, H., Gu, C., Luo, F., Ding, W. & Liu, X. Optimization of lightweight task offloading strategy for mobile edge computing based on deep reinforcement learning. *Futur. Gener. Comput. Syst.* **102**, 847–861. <https://doi.org/10.1016/j.future.2019.07.019> (2020).
39. Hossain, M. D. et al. Fuzzy decision-based efficient task offloading management scheme in multi-tier MEC-enabled networks. *Sensors* **21**, 1484. <https://doi.org/10.3390/s21041484> (2021).
40. Nguyen, D. C., Pathirana, P. N., Ding, M. & Seneviratne, A. Privacy-preserved task offloading in mobile blockchain with deep reinforcement learning. *IEEE Trans. Netw. Serv. Manage.* **17**, 2536–2549. <https://doi.org/10.1109/TNSM.2020.3010967> (2020).
41. Samy, A., Elgendy, I. A., Yu, H., Zhang, W. & Zhang, H. Secure task offloading in blockchain-enabled mobile edge computing with deep reinforcement learning. *IEEE Trans. Netw. Serv. Manage.* **19**, 4872–4887. <https://doi.org/10.1109/TNSM.2022.3190493> (2022).
42. Xu, X. et al. Trust-aware service offloading for video surveillance in edge computing enabled Internet of vehicles. *IEEE Trans. Intell. Transp. Syst.* **22**, 1787–1796. <https://doi.org/10.1109/ITITS.2020.2995622> (2021).
43. Wei, D. et al. Personalized privacy-aware task offloading for edge-cloud-assisted industrial Internet of things in automated manufacturing. *IEEE Trans. Industr. Inf.* **18**, 7935–7945. <https://doi.org/10.1109/TII.2022.3159822> (2022).
44. Lingayya, S., Jodumutt, S. B., Pawar, S. R., Vylala, A. & Chandrasekan, S. Dynamic task offloading for resource allocation and privacy-preserving framework in Kubeedge-based edge computing using machine learning. *Cluster Comput. (Early Access)*[SPAC E]<https://doi.org/10.1007/s10586-024-04420-8> (2024).
45. Dai, X. et al. A learning-based approach for vehicle-to-vehicle computation offloading. *IEEE Internet Things J.* **10**, 7244–7258. <https://doi.org/10.1109/JIOT.2022.3228811> (2023).
46. Zhang, T., Li, Y. & Chen, C. P. Edge computing and its role in industrial Internet: Methodologies, applications, and future directions. *Inf. Sci.* **557**, 34–65. <https://doi.org/10.1016/j.ins.2020.12.021> (2021).
47. Sabella, D. et al. MEC security: Status of standards support and future evolutions. *ETSI White Paper* **46**, 1–26 (2023).
48. Wang, C. et al. The security and privacy of mobile-edge computing: An artificial intelligence perspective. *IEEE Internet Things J.* **10**, 22008–22032. <https://doi.org/10.1109/JIOT.2023.3304318> (2023).
49. Shannon, C. E. A mathematical theory of communication. *Bell Syst. Tech. J.* **27**, 379–423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x> (1948).
50. Hu, M. et al. Learning driven computation offloading for asymmetrically informed edge computing. *IEEE Trans. Parallel Distrib. Syst.* **30**, 1802–1815. <https://doi.org/10.1109/TPDS.2019.2893925> (2019).
51. Dong, J., Wan, S. & Chen, S. Fuzzy best-worst method based on triangular fuzzy numbers for multi-criteria decision-making. *Inf. Sci.* **547**, 1080–1104. <https://doi.org/10.1016/j.ins.2020.09.014> (2021).
52. Detti, P. A new upper bound for the multiple knapsack problem. *Comput. Operat. Res.* **129**, 105210–105220. <https://doi.org/10.1016/j.cor.2021.105210> (2021).
53. Chakraborty, S. & Mazumdar, K. Sustainable task offloading decision using genetic algorithm in sensor mobile edge computing. *J. King Saud Univ. Comput. Inform. Sci.* **34**, 1552–1568. <https://doi.org/10.1016/j.jksuci.2022.02.014> (2022).

Acknowledgements

This work was partly supported by the National Natural Science Foundation of China (No. 62301425), the Scientific Research Program Funded by Education Department of Shaanxi Provincial Government (No. 23JP164), the Graduate Innovation Fund of Xi'an, an University of Posts and Telecommunications (No. CXJJZL2023025) and Special Funds for Construction of Key Disciplines in Universities in Shaanxi.

Author contributions

X. J., S. Z. and Y. D. established the system model and designed the offloading algorithm. X. J. designed the experiments. S. Z. and Y. D. conducted the experiments. X. J. and Z. W. analysed the results. All authors reviewed the manuscript.

Competing interests

Te authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to X.J.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2024